

ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ
ΠΡΟΓΡΑΜΜΑ ΜΕΤΑΠΤΥΧΙΑΚΩΝ ΣΠΟΥΔΩΝ
ΤΜΗΜΑΤΟΣ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

Two-way learning for TD Repayment Prioritization: A human-centric approach

Πτυχιακή Εργασία

της

Κανίδου Ελिसάβετ Περσεφόνη

Θεσσαλονίκη, Ιούλιος 2024

Περίληψη

Αυτή η πτυχιακή εργασία πραγματεύεται μια ανθρωποκεντρική προσέγγιση για την ιεράρχηση της αποπληρωμής τεχνικού χρέους (TD). Το TD περιλαμβάνει αντισταθμίσεις που γίνονται από προγραμματιστές για να επιταχύνουν την παράδοση λογισμικού, αυξάνοντας πιθανώς το μελλοντικό κόστος συντήρησης και ανάπτυξης. Η έρευνα στοχεύει να αναπτύξει ένα μοντέλο μηχανικής μάθησης που προβλέπει ποιες κλάσεις κώδικα είναι πιο πιθανό να απαιτούν αναδιαμόρφωση, λαμβάνοντας υπόψη τις ανάγκες και τις προτεραιότητες των ομάδων ανάπτυξης.

Για να επιτευχθεί αυτό, αναλύθηκαν διάφορα εργαλεία και τεχνικές για την ανίχνευση και τη διαχείριση του TD. Το μοντέλο μηχανικής μάθησης που επικυρώθηκε χρησιμοποιώντας δεδομένα πραγματικού κόσμου, έδειξε την αποτελεσματικότητά του όσον αφορά την ιεράρχηση της αποπληρωμής TD. Επιπλέον, το μοντέλο αυτό ενσωματώθηκε σε μια εφαρμογή, παρέχοντας ένα πρακτικό εργαλείο για τις ομάδες ανάπτυξης για τη διαχείριση και την ιεράρχηση του τεχνικού χρέους τους με πιο αποτελεσματικό τρόπο. Το μοντέλο εξηγεί επίσης γιατί μια συγκεκριμένη κλάση χρειάζεται ανακατασκευή, επιτρέποντας στους χρήστες να παρέχουν ανατροφοδότηση συμφωνώντας ή διαφωνώντας με τις προτάσεις του μοντέλου. Αυτή η ανατροφοδότηση χρησιμοποιείται στη συνέχεια για την επανεκπαίδευση και τη συνεχή βελτίωση του μοντέλου.

Τα βασικά ευρήματα δείχνουν ότι η εφαρμογή αυτού του μοντέλου μπορεί να βελτιώσει σημαντικά την ποιότητα και τη συντηρησιμότητα του λογισμικού, ενώ μειώνει το μελλοντικό κόστος συντήρησης και ανάπτυξης.

Λέξεις Κλειδιά:

Τεχνικό χρέος, ανακατασκευή, μηχανική μάθηση, ιεράρχηση προτεραιοτήτων, διαχείριση χρέους.

Abstract

This thesis deals with a human-centric approach to prioritizing technical debt (TD) repayment. TD involves trade-offs made by developers to expedite software delivery, potentially increasing future maintenance and development costs. The research aims to develop a machine learning model that predicts which code classes are most likely to require refactoring, considering the needs and priorities of development teams.

To achieve this, various tools and techniques for detecting and managing TD were analyzed. The machine learning model validated using real-world data and demonstrated its effectiveness in prioritizing TD repayment. Additionally, this model was integrated into an application, providing a practical tool for development teams to manage and prioritize their technical debt more effectively. The model also explains why a particular class needs refactoring, allowing users to provide feedback by agreeing or disagreeing with the model's suggestions. Then this feedback is used to retrain and improve the model continuously.

The key findings indicate that applying this model can significantly enhance software quality and maintainability while reducing future maintenance and development costs.

Keywords:

Technical debt, refactoring, machine learning, prioritization, debt management.

Acknowledgements

I would like to extend my heartfelt thanks to my supervising professor and his team for their invaluable guidance and support. Additionally, I am deeply grateful to my family for their continuous encouragement and support.

Contents

Contents	vi
1 Introduction	1
1.1 Problem – Importance of the Topic	1
1.2 Purpose - Objectives	1
1.3 Questions – Hypotheses	1
1.4 Contribution	2
1.5 Key Terminology	2
1.6 Structure of the Study	2
2 Literature Review – Theoretical Background	3
2.1 Refactoring tools	3
2.2 Analysis of Code Restructuring Detection Tools	4
2.2.1 Summary of Findings	6
2.3 A Comprehensive Examination of Tools for Technical Debt in Codebases	6
2.4 Technical Debt	8
2.5 Machine Learning in Technical Debt Management	8
2.6 Performance Metrics	8
2.7 Xai	10
2.7.1 XAI- SHAP	10
2.7.2 Xai-Lime	12
2.7.3 Comparison	13
3 Methodology	15
3.1 For the idea	15
3.2 Dataset Overview and Preprocessing:	15
3.3 Balanced Dataset Composition and Model Selection	16
3.3.1 Introduction.....	16
3.3.2 Phase 1	17
3.3.3 Phase 2	22
3.4 Parameter Adjustment for KNN	23
3.5 Parameter Evaluation for Optimal Parameters	26
3.6 Detailed Description of Scripts for Predictive Analysis and Explanations	26
3.7 Python	28

3.8 Validation with Python	34
3.8.1 Comparative Analysis and Benefits of New Approach.....	35
3.8.2 Code.....	37
4 Integration with debt clock platform	39
4.1 For the Application	39
4.1.1 Back end	42
4.1.2 The model in the platform	50
5 Results	52
5.1 Python Model - Using Data from All Projects Except the One to Be Predicted	52
5.2 Python Model - Using Data from the Same Project	53
5.3 Validation with Python Model - Results	54
6 Conclusion	59
6.1 Summary and conclusions	59
6.2 Limitations of the Research	59
6.3 Interpretability Challenges:	59
6.4 Data Limitations:	59
6.4.1 Generalization of Models:	60
6.4.2 Overfitting:.....	60
6.4.3 Scalability:	60
6.4.4 Future Improvements Required:	60
6.5 Future Expansion	61

List of Figures

Figure 1. Model 1 - Accuracy of Different Kernel Functions Across Various Neighbors and Distances	18
Figure 2. Model 2 - Accuracy of Different Kernel Functions Across Various Neighbors and Distances	19
Figure 3. Model 3 - Accuracy of Different Kernel Functions Across Various Neighbors and Distances	20
Figure 4. Dashboard of application	39
Figure 5. Top Contributors in Code Quality and Refactoring.....	40
Figure 6. Heatmap of High – TD Probability per Class	41
Figure 7. SHAP – HIGH TD interest Risk.....	41
Figure 8. SHAP - force plot	42
Figure 9. Heatmap of High Refactoring Probability per Class	50
Figure 10. Lime plot	51
Figure 11. Similar Past Cases of Performed Refactorings.....	51

List of Tables

Table 1. Results of models in phase 2	22
Table 2. Results of Bookkeeper project.....	54
Table 3. Results of Hudi project.....	55
Table 4. Results of Incubator-Celeborn project	56
Table 5. Results of ShardingSphere-ElasticJobproject.....	57

1 Introduction

This thesis explores the prioritization of technical debt repayment through a human-centric approach. Technical debt refers to the trade-offs developers make to expedite software delivery, which may increase future maintenance and development costs. Managing and repaying technical debt is crucial for maintaining software quality and preventing significant future issues.

1.1 Problem – Importance of the Topic

This study examines the significance of technical debt and its management. An increase in technical debt can lead to software quality issues, higher maintenance costs, and reduced productivity for development teams. Repaying technical debt is a critical issue that requires careful planning and strategic decisions. This study focuses on using analysis tools and machine learning techniques to prioritize technical debt repayment in a human-centric manner, considering the needs and priorities of development teams.

1.2 Purpose - Objectives

The purpose of this thesis is to develop a method for prioritizing technical debt repayment that accounts for the needs of the people working on the code. The objectives include analyzing tools and techniques used for detecting and managing technical debt, developing a machine learning model that predicts which code classes are more likely to need refactoring, and applying this method to real data to evaluate its effectiveness.

1.3 Questions – Hypotheses

Which code classes are most likely to need refactoring?

How can analysis tools improve the process of prioritizing technical debt repayment?

What is the impact of this prioritization on the overall quality and maintainability of the software?

1.4 Contribution

The contributions of this study include:

Developing a machine learning model for predicting technical debt.

Applying and evaluating the model on real software data.

Providing a human-centric framework for prioritizing technical debt repayment.

Comparative analysis of the performance of different technical debt detection tools.

1.5 Key Terminology

Technical Debt: The compromises made during software development to achieve quick delivery, which may increase future maintenance and development costs.

Refactoring: The process of improving the structure of existing code without changing its external behavior.

Machine Learning: A subset of artificial intelligence that uses algorithms and statistical models to enable computers to improve their performance on specific tasks through experience.

1.6 Structure of the Study

This thesis is organized as follows:

Chapter 2: Presentation of tools and techniques for detecting and managing technical debt.

Chapter 3: Development and evaluation of the machine learning model for predicting technical debt.

Chapter 4: Application of the proposed method to real data and analysis of the results.

Chapter 5: Conclusions and suggestions for future research.

2 Literature Review – Theoretical Background

2.1 Refactoring tools

The on-going software technology advancement has urged developers to find ways so as to increase productivity and at the same time the quality of their code writing. Thus, the tools for code restructuring are becoming invaluable to this effort. With features that assist developers to simultaneously arrange, improve and streamline their codebase, applications as ReSharper, Eclipse JDT, CodePro Analytix, and Understand are just some on the long list of applications that belong in this category. With each application providing a variety of features that aim to satisfy the constantly increased requirements of contemporary software developers (Murphy-Hill & Black, 2008).

While examining thoroughly the use of those tools, one may conclude that different developing fields utilize different tools for greater efficiency. More specifically, ReSharper, a JetBrains tool, is mainly used by C# and VB.NET developers, while ReSharper which is a feature-rich code reorganization tool that works efficiently with Microsoft Visual Studio (JetBrains, 2024). Additionally, the latter offers a comprehensive code inspection capability and navigation support. Despite its efficiency that enables developers to write clear and manageable code, there have been detected some drawbacks as far as ReSharper's licensing requirements are concerned but also regarding its performance overheads inside the Visual Studio ecosystem.

As far as the examination of the Eclipse Java Development Tools (JDT) they offer to developers a wide range of Java-specific code restructuring features (Eclipse Foundation, n.d.). Apart from fitting perfectly with the Eclipse development environment, those tools, provide also a strong support for Java developers. In the case of specific large-scale projects, the tool may present some performance bottlenecks that will demand more careful and to the point optimization techniques. What is more, for the more efficient handling of the application, it will be necessary for developers to dedicate more time and effort to adjust to the subtle nuances of the Eclipse environment.

CodePro Analytix, a product deriving from Google genius, comes right on time when there is a great need for relentlessly enhancing the quality and maintainability of codes (Google, n.d.). It is a utility suite of software engineering analyzing and improving

codebases written in C++, Java, and C#. It has all the capability to seamlessly integrate with trendy IDEs, such as Eclipse, making it very useful in maintaining the quality of the code and avoiding technical debt. However, the need to license the tool and the inherent difficulty around some of the functionalities will be identifiable barriers to the take-up and use of the tool.

Hence, as far as the Understand plug in tool, is concerned it becomes the panacea of knowledge whereby the developer enjoys an exquisite and rich suite of code structures and metrics for any number of programming languages (Springer, n.d.; ResearchGate, 2018). More than this, the tool offers a large codebase that may be properly familiarized and navigated, thus helping developers to develop informed and effective decisions toward enhancing code quality and maintainability (Cambridge University Press, n.d.). However, Understand is not easy to use, and its inherent complexity calls for intensively focused efforts expected from the end-users in the direction of proficiency acquisition (IEEE Xplore, 2008).

In conclusion, for the selection of any effective code restructuring tool, many factors will have to come into play: the demands of the development project at hand, the experience and knowledge of the development team with regards to the selected set of tools, and the overall objective of the development project or scheme. Despite all of these tools' potential impediments and complexities, there can be no equivocation that they play an indispensable role in enhancing both the quality and quantity of software production and, thus, are essential facets of the modern software engineering paradigm.

2.2 Analysis of Code Restructuring Detection Tools

In turn, software refactoring is an essential process in software development, as it allows the developer to enhance the design and quality of the source code (Mens & Tourwé, 2004; Fowler, 2018). This process of refactoring enables the developer to reorganize the existing code to increase the maintainability, readability, and scalability of software systems without observable behavior change of this code. However, an understanding of the effect of these code changes imposed by refactoring is one of the prerequisites of software maintenance and evolution (Dig, Manzoor, Johnson, & Nguyen, 2008; SonarQube, n.d.).

In this concern, many tools have been introduced that subject code restructurings to changes regarding detection and analysis in a fully automated manner (IEEE Xplore, 2023; Kamiya, 2015). Since MiningRefactoring is an open-source tool to get insight into the change history of software repositories to detect those changes that indicate some code restructurings, RefactoringMiner uses changes that happened in the source code present in many different versions. It offers the functionality of the analysis of the history of a repository through detecting restructurings and their impacts, and through using such information, the development teams using RefactoringMiner can know how their code base has gone through changes for getting enlightenment on whether or where to refactor (ResearchGate, 2022; Chatzigeorgiou et al., 2020; CEUR-WS, 2019).

These tools include RefFinder and many others, which have been equipped with advanced methodologies for recognizing code movements and code changes in software repositories. RefFinder boasts a flexible approach in that, among other recognized code restructurings, it can handle those executed across various types of software projects; it becomes more adaptable and, hence, more user-friendly (Kamiya, 2015; Chatzigeorgiou et al., 2020).

RefacDroid focuses on restructuring detection within the Java code of Android applications. Analyzing the changes to the Java source code version history opens the view of the Android project evolution to developers, which would otherwise enable carrying out excellent and consistent code maintenance (Oliveira, Mota, & Reis, 2024).

While RefFinder uses pattern-based and analytical methods to detect those cases in software repositories, the occurrences of code restructuring were also detected in both categories. The methodology implemented with the added feature will contribute to improving the restructuring detection accuracy in RefFinder, making it a substantial research contribution that can be applied in software maintenance and evolution studies (Kamiya, 2015).

Essentially, JDeodorant acts as an open-source software designed for the analysis of Java code and, therefore, is able to list a vast deal of some determined "code smells" and to propose given code restructurings to improve this code structure. JDeodorant adds advantages and improvements to the software quality in terms of maintainability due to

clean code and design-pattern-based features by detecting and removing existing code smells (Tsantalis, N., et al., 2008).

2.2.1 Summary of Findings

All of the above tools have different functionalities and approaches for the code analysis and detection of restructuring. The most optimum choice can be decided based on the given factors within the project, the development environment, and individual preferences. All these limitations aside, all the tools covered so far form an indispensable part of modern software development because they help the developer maintain, evolve, and improve the quality of the code. In summary, the significance of the detection tools for code restructuring tends to repeat itself with modern practices in software engineering. These automate the process of identifying the changes with perfect code analysis. Eventually, the software developer will have a chance to manage the complexity of software better and sustain the changes for a more extended period. Most projects are growing in complexity in line with the evolution and growth of various effortless software projects. With the maintenance of the quality of the code and innovation of the software, there will always be a need for code restructuring detection tools for refinements.

2.3 A Comprehensive Examination of Tools for Technical Debt in Codebases

Technical debt is quite a critical issue as far as software programming is concerned. It involves ways of coding that, in a way, offer the same results for the present moment but later have the ability to cause problems in code and even raise maintenance costs (Chatzigeorgiou et al., 2020). This report examines some available tools for technical debt identification and management in a software code base (SonarQube, n.d.).

The tool is one of the most important in this area, and it thoroughly analyzes the code quality. It focuses on visualization and detection of technical debt (Oliveira, Mota, & Reis, 2024). This set of indicators of measuring quality consists of code duplications, complex methods, and any violation of programming principles and best practices (IEEE

Xplore, 2008). SonarQube allows development teams to identify and reduce technical debt, improve code quality, and maintain its quality (SonarQube, n.d.)

In that direction, advanced code quality measurements are made by Code Climate, too; thus, technical debt is highlighted just like SonarQube. It-based multifaceted metrics enable the development teams to pinpoint the problems with code very accurately and take strategic action to deal with the technical debt (Oliveira, Mota, & Reis, 2024).

PMD is effectively a Java code analyzer, it does great work detecting your quality problems-technical debt. PMD amicably provides practitioners with hands-on recommendations to improve your code, reduce technical debt, and make it possible for the development teams to optimize your codebase very much to a practical finish (ResearchGate, 2023).

FindBugs is a tool for Java-based projects, and ideally, most of the quality issues, along with technical debt, would be highlighted by the tool. Given its deep analysis, the ways to optimize the code and reduce technical debt will be shown to development teams (IEEE Xplore, 2008).

At last, Checkstyle is used as a mature tool to analyze Java code and embed why the code fails to meet coding standards and best practices related to making the technical debt. Its deep analysis allows the development team to strictly obey the coding conventions and reduce the technical debt prevalence with its keen features (IEEE Xplore, 2008).

All of the aforementioned tools have a broad range of characteristics concerning the analysis of technical debt within the codebases of their respective supporting software. With their proper use, development teams can quickly analyze problems with the code quality and thus fix them appropriately so that the growth of technical debt is diminished to a more significant extent; hence, to a more prominent extent, code maintainability and efficiency can be improved.

2.4 Technical Debt

Technical debt is the compromises made in software development, which will help in quick delivery but might increase future maintenance and development costs. Managing and repaying technical debt are essential for software quality and avoiding major future troubles. The metaphor used regarding technical debt is near to financial debt; if taken, the cost of repayment grows over time when not repaid immediately (Tom, E., Aurum, A., & Vidgen, R., 2013; Kruchten, P., Nord, R. L., & Ozkaya, I., 2012).

2.5 Machine Learning in Technical Debt Management

Machine learning is a subfield of artificial intelligence; it is the ability of computers to learn some tasks and improve on it based on experience with the aid of an appropriate algorithm and statistical model. In the case of technical debt management, machine learning will be applied to predict the classes of code that are more likely to need refactoring, hence increasing the efficiency in repaying this debt (Bavota, G., Russo, B., Oliveto, R., & Poshyvanyk, D., 2016; Mensah, S., Yang, Y., & Du, W., 2018).

2.6 Performance Metrics

Model performance for the prediction of technical debt is evaluated with consideration of several important metrics that describe the effectiveness and robustness of the models:

Accuracy: Accuracy represents the fraction of correct predictions the model makes out of all the predictions. Though conceptually simple, accuracy might not be the ideal metric for imbalanced class problems (Powers, D. M., 2011).

$$Accuracy = \frac{True\ Positives + True\ Negatives}{Total\ Predictions}$$

Precision: Precision quantifies the accuracy of optimistic predictions made by the model. This can be calculated using the number of positive results that turned out to be accurate, divided by all the positive results predicted by the model (Saito, T., & Rehmsmeier, M., 2015).

$$Precision = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Positives\ (FP)}$$

Recall: Sensitivity, also called recall, is a measure that shows how well the model has identified all relevant instances. It is measured in the number of accurate positive results by the dataset's actual positives (Powers, D. M., 2011).

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

F1 Score: The F1 score is the harmonic mean of precision and recall, making it a single metric that trades both concerns against each other. It is particularly useful when the class distribution is imbalanced (Powers, D. M., 2011).

$$F1 = 2 * \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F2 Score: The F2 score is the same as an F1 score, but more weights are put on recall than precision. Thus, this is useful in conditions where false negatives are costlier than false positives (Saito, T., & Rehmsmeier, M., 2015).

$$F2 = \frac{(1 + 2)^2 \text{ Precision} \cdot \text{Recall}}{2^2 \text{ Precision} + \text{Recall}}$$

ROC-AUC Score: The ROC-AUC (Receiver Operating Characteristic - Area Under Curve) score indicates the ability of a model to separate the classes of an objective variable. A value closer to 1 in an ROC-AUC implies that it has a better score at separating positive from antagonistic courses (Powers, D. M., 2011).

These metrics provide a proper understanding of the goodness or weakness of the model with predictive capability. With these metrics, the evaluation process of machine learning models becomes more detailed and robust.

2.7 Xai

Explainable Artificial Intelligence (XAI) has become relevant as machine learning models have proliferated into every sphere of society—from health care to finance and more (Adadi, A., & Berrada, M., 2018). While such models exhibit excellent predictive performance, the intrinsic complexity makes it hard to understand why a model makes certain decisions (Doshi-Velez, F., & Kim, B., 2017). This leads to problems involving opacity in high-stake applications where decisions are made over human lives or significant societal consequences (Guidotti, R., et al., 2018). The increasing demand for machine learning has, consequently, led scientists to design a wide range of XAI methods that give orientation to the machine learning models (Ribeiro, M. T., Singh, S., & Guestrin, C., 2016).

SHAP and LIME are two significant techniques used within the XAI domain. Though both LIME and SHAP are targeting offering insights into machine learning models on the basis of their predictions, both stem from different directions and are founded on other strengths as well as weaknesses (Molnar, C., 2019).

2.7.1 XAI- SHAP

SHAP framework is based and founded on game theory; namely, one of the cooperative game theories attributes a value to every feature of the model as SHAP as represented by contribution engraved on a prediction. These values also give the best of both worlds. They offer local and global interpretation such that the stakeholders can have a sense of any individual predictions and also the overall behavior of the model (Lundberg, S. M., & Lee, S.-I., 2017). For any individual prediction, LIME generates an interpretable explanation locally by approximating a surrogate model of high fidelity around that data point in a much more local approach. While LIME is model-agnostic and applies to any black-box model, a small data set or data sets with very complex relationships between features might prove challenging (Lundberg, S. M., et al., 2018).

SHAP is one of the most robust methodologies for explaining machine learning-derived predictions within the wide variety of frameworks that make up XAI. SHAP swings around the SHAP value concept. It serves as the foundation for explaining the contribution of each feature within the model (Lundberg, S. M., & Lee, S.-I., 2017). Therefore, these values would capture the contribution that an individual could expect

upon average for a feature having on the model's prediction about that data instance. Worth noting here is that SHAP values can contain both positive and negative magnitudes. The sign of the magnitude tells one the direction of impact of that particular feature upon the model's prediction.

SHAP offers several advantages over other XAI techniques, including local interpretability—that is, it allows explaining model predictions locally while enabling stakeholders to understand the reasons for the decisions made by one for every data point. As part of this, they also ensure that SHAP values are invariant concerning the ordering of feature consideration during computation, which makes any such explanation more coherent and reproducible. Rooted in the concept of cooperative game theory, SHAP values impose a guide principle methodology for the equitable distribution of credits across different features relevant to the output gathered through it so that one knows how the predictive power is to be shared around.

While SHAP, although very useful, is not without any limitations. Computational complexity, especially where high dimensionality and complex models are concerned, may imply that the computation of SHAP values is inefficient. At the same time, the interpretation of SHAP values can also be challenging, particularly for business stakeholders who might not have deep knowledge of machine learning/XAI. Lastly, being a relatively new framework, active research and development in enhancing and optimizing its use are underway to overcome present limitations and extend its capability. It is a precious tool for machine learning's inner mystery unraveling, and nuanced and consistent, theoretically grounded explanations of the machine learning model are at its heart.

Overall, even though SHAP is computationally demanding with challenges in its interpretability, current research alleviates some of its limitations. It thus marks it as an indispensable asset in the field of XAI. As XAI continues to advance, SHAP is similarly very well placed to provide stakeholders with the understanding and confidence to deploy machine learning models that really work and do so in a responsible way.

2.7.2 Xai-Lime

Recently, an excellent addition to the domain of XAI is LIME or Local Interpretable Model-Agnostic Explanations; it is basically applied for interpreting classifier predictions. This method explains different machine learning models locally—more precisely, every prediction made by these models is presented (Ribeiro, M. T., Singh, S., & Guestrin, C., 2016). LIME operates under the principle of training a linear model whose coefficients are proportional to each feature. In this way, quantifiably, each feature gets represented as to what magnitude of importance they are during the predictions the performing model returns (Ribeiro, M. T., Singh, S., & Guestrin, C., 2016). Operated on perturbation and reevaluation, LIME pays more notable attention to features that dramatically change a model's predictions, thus affording substantial weight while classifying features that do not have as much effect as negligible.

Famous though it remains, with lots of business utility derived from its application, LIME is not without overall challenges and limitations. One of the significant drawbacks comes when working with a small data set, as LIME evaluates all features as unimportant due to lesser samples or data points. Every data set feature result is also unique, and LIME cannot capture commonalities between different data sets and their features. Such limitations also bring about the need for alternative procedures grounded in other methods, for instance, utilizing SHAP (SHapley Additive exPlanations) values that incorporate a different explanation of feature importance.

Besides, the usefulness of LIME lies beyond its capabilities, scalability, and adaptability in various problem domains. Regarding large-scale data analysis, LIME allows the advantages of the scalability that is obtained due to the techniques of data mining. It will enable for explaining complex models based on big datasets. Moreover, while other methods, for example, ANCHORS, may be more suitable in categorical or discrete feature sets, LIME has relevant applications on datasets with continuous features since information may be lost due to discretization (Ribeiro, M. T., Singh, S., & Guestrin, C., 2016).

The further development of LIME and related methods will greatly benefit the development of the XAI field. Already, many citations and the comprehensiveness evaluation just make LIME a building block of machine learning model transparency and

interpretability. Just by focusing on practical improvements and bridging inherent gaps, researchers could apply LIME in most applications much more effectively.

In these terms, LIME is that step. It views a model agnostically for local explainability. Although not without its weaknesses, LIME is very scalable, straightforward, and highly flexible. As such, it would be an excellent addition to the set of tools that make transparent and explainable machine learning in the move towards having methods that would make AI more understood and trusted. Future research will undoubtedly be conducted building on these strengths, working around the weaknesses, so that LIME becomes a powerful, perfectly transparent, accountable tool with all potential (Ribeiro, M. T., Singh, S., & Guestrin, C., 2016).

2.7.3 Comparison

In summary, while LIME and SHAP have both provided significant extensions to the field of explainable artificial intelligence (XAI), respectively, each manifests unique features regarding both strengths and limitations. Developed from the gamut of cooperative game theory and computation of SHAP value, SHAP creates refined information on the contribution of different features to model predictions (Lundberg, S. M., & Lee, S.-I., 2017). The fact that SHAP is locally interpretable and theoretically grounded makes it robust machinery in the understanding of machine learning models. However, the need to strive to the fullest realization of the potential of local interpretability remains, albeit for challenges such as computational complexity and interpretational hurdles (Lundberg, S. M., et al., 2018).

LIME, on the other hand, is locally interpretable and model-agnostic, thereby able to generate explanations for individual predictions (Ribeiro, M. T., Singh, S., & Guestrin, C., 2016). Although it is general, simple, and scalable to almost any dataset or feature composition, it is only used to address limited heterogeneity in the features and small data. However, new active improvements in LIME methodology may bring hope in sorting these issues out and making LIME strong enough to enhance holistically across the board on specific problems under the umbrella of XAI.

Moving through the XAI landscape, practitioners/contributors and researchers must act responsibly by clearly constraining the actual needs and limitations of their work to correctly choose the most suitable technique. Although SHAP provides a broad scope of explaining the model's behavior, LIME provides local, which may be more valuable in some realization cases. Ultimately, the continued improvement and unification of these two methodologies work toward the common goal of making AI systems more transparent, more understandable, and thereby wielded by stakeholders appropriately and ethically in all domains.

3 Methodology

3.1 For the idea

In this thesis, the developed AI model is meant to deal with the significant problem of technical debt in software development by predicting which code classes will most likely need refactoring. The developed model uses machine learning techniques to analyze different code metrics and patterns, learning from historical data to make accurate predictions. The model works in coherence with Explainable AI techniques such as SHAP and LIME, which provide transparent and interpretable explanations for its predictions. This allows developers to understand why certain classes are marked for refactoring.

The model isn't static; it includes a feedback mechanism where developers can agree or disagree with its suggestions. This feedback loop is crucial for the model's continuous improvement, enabling it to adapt to evolving coding practices and the specific needs of the development team. Overall, the AI model aims to help development teams proactively manage technical debt, maintain high code quality, and reduce long-term maintenance and development costs.

3.2 Dataset Overview and Preprocessing:

All these steps were well-thought-out at the very inception, considering the dataset for the training of a model since it is largely agreed upon that the quality of the dataset can be represented through the performance of a model. The dataset mainly comprises projectName, SHA, file, DIT, CC, LCOM, MPC, NOM, RFC, DAC, NOCC, CBO, SIZE1, and REFACTORED. At the preprocessing stage, there was a need to make a strategic decision while trying to keep the integrity of the dataset. For example, it was explicitly decided that if a class is included in the TRUE segment of the dataset, it should not be randomly assigned FALSE in another commit. This approach was taken to keep the class representation coherent and consistent within the dataset.

Adding to the above, in order to make the dataset more reliable and free from redundancy, all duplicate entries in this dataset were scrutinized and removed carefully. This was done to ensure that no class in the dataset was redundant while allowing a few exceptions where several commits were relevant to the same class. For this, the following code snippet was used to identify duplicate elements within the 'file' column of the dataset:

```
R
# Detecting duplicate elements in the 'file' column of the dataset
duplicates <- duplicated(dataset[c("file")])
unique_entries <- dataset[!duplicates]
Save to grepper
This preprocessing step not only streamlined the dataset but also optimized its utility for subsequent model training endeavors.
```

3.3 Balanced Dataset Composition and Model Selection

The creation and selection of appropriate dataset configurations for model training required careful deliberation and experimentation to achieve optimal performance outcomes. This process involved developing and evaluating three distinct models to determine the most suitable approach for our specific objectives (Bishop, C. M., 2006; Murphy, K. P., 2012).

3.3.1 Introduction

The goal of this study is to evaluate the performance of three different K-Nearest Neighbors (KNN) models applied to a dataset where class consistency is crucial—if a class is true in one commit, it cannot be false in another. The dataset is split into training and testing subsets with a 70-30 ratio. This research aims to find the optimal KNN configuration by exploring various preprocessing techniques, kernel functions, and distance metrics (Cover, T. M., & Hart, P. E., 1967; Fix, E., & Hodges, J. L., 1951).

The research took place in two phases. Development and testing of three models continued for the first phase in the attempt to find the most appropriate approach for our aims. Each model adopted particular steps of data pre-processing concerning issues such as duplicate entries and class imbalance. The second phase encompassed another three models designed to re-form further and fortify the initial approaches. This more elaborate preprocessing and strict attention to the chosen criterion in the second-phase model safeguards that the data are robust enough to uphold a greater degree of precision and consistency in the final predictive outcome.

3.3.2 Phase 1

The study is divided into three primary models, each incorporating specific data preprocessing steps to address issues such as duplicate entries and class imbalance. The steps are as follows:

First Model:

- ❖ **Condition:** Ensures that if a class is true in one commit, it cannot be false in another.
- ❖ **Data Split:** The dataset is split into 70% for training and 30% for testing.
- ❖ **Note:** The set of "yes" is smaller.

Second Model:

- ❖ **Condition:** Same as the first model.
- ❖ **Preprocessing:** Identifies and removes duplicate entries in the "no" dataset. Constructs a balanced training and testing dataset by considering the size of the smaller class (either "yes" or "no").

Third Model:

- ❖ **Condition:** Same as the first model.
- ❖ **Preprocessing:** Identifies and removes duplicate entries in both the "yes" and "no" datasets.

Each model is evaluated using repeated cross-validation to measure performance metrics such as accuracy and kappa statistics across various combinations of k values (number of neighbors), distance metrics, and kernel functions. The kernel functions explored include Gaussian, Triangular, Rectangular, Epanechnikov, and Optimal.

3.3.2.1 Results

First Model:

➤ Performance:

- ❖ **Accuracy:** 0.8797
- ❖ **Kappa:** 0.7593
- ❖ **Accuracy Standard Deviation:** 0.0168
- ❖ **Kappa Standard Deviation:** 0.0336
- ❖ **Observation:** All kernel functions (Gaussian, Triangular, Rectangular, Epanechnikov) yielded identical performance metrics, indicating consistent behavior across different kernels and distance metrics.

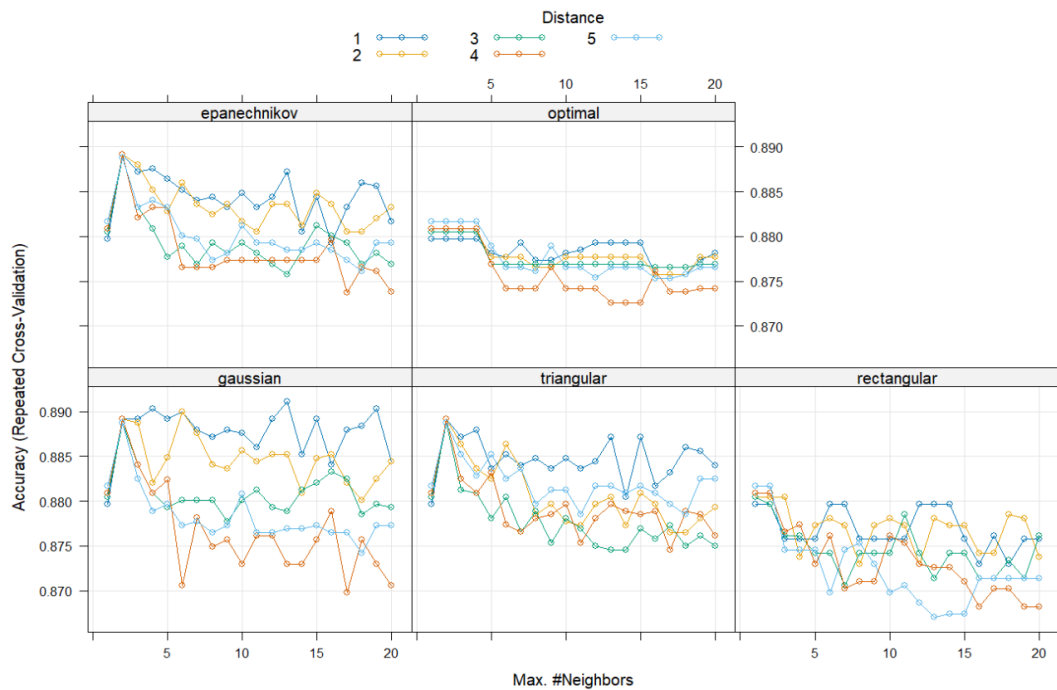


Figure 1. Model 1 - Accuracy of Different Kernel Functions Across Various Neighbors and Distances

Second Model:

➤ Performance:

- ❖ **Accuracy:** 0.8856
- ❖ **Kappa:** 0.7711
- ❖ **Accuracy Standard Deviation:** 0.0138
- ❖ **Kappa Standard Deviation:** 0.0276

- **Observation:** The model showed improved accuracy and kappa compared to the first model. The inclusion of the Optimal kernel function maintained the same performance metrics, demonstrating the effectiveness of removing duplicates in the "no" dataset and balancing the dataset for training.

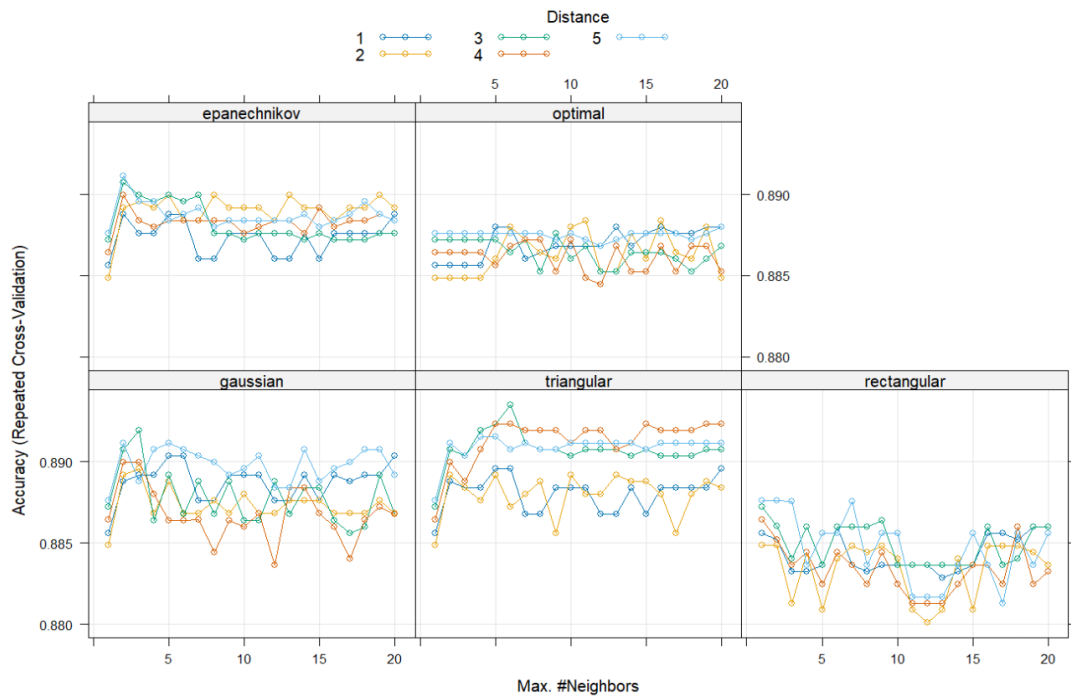


Figure 2. Model 2 - Accuracy of Different Kernel Functions Across Various Neighbors and Distances

Third Model:

➤ Performance:

- ❖ **Accuracy:** 0.8018
- ❖ **Kappa:** 0.6036
- ❖ **Accuracy Standard Deviation:** 0.0226
- ❖ **Kappa Standard Deviation:** 0.0447

- **Observation:** This model exhibited a decline in performance metrics compared to the first and second models. The removal of duplicates in both "yes" and "no" datasets may have introduced variability, impacting the model's effectiveness.

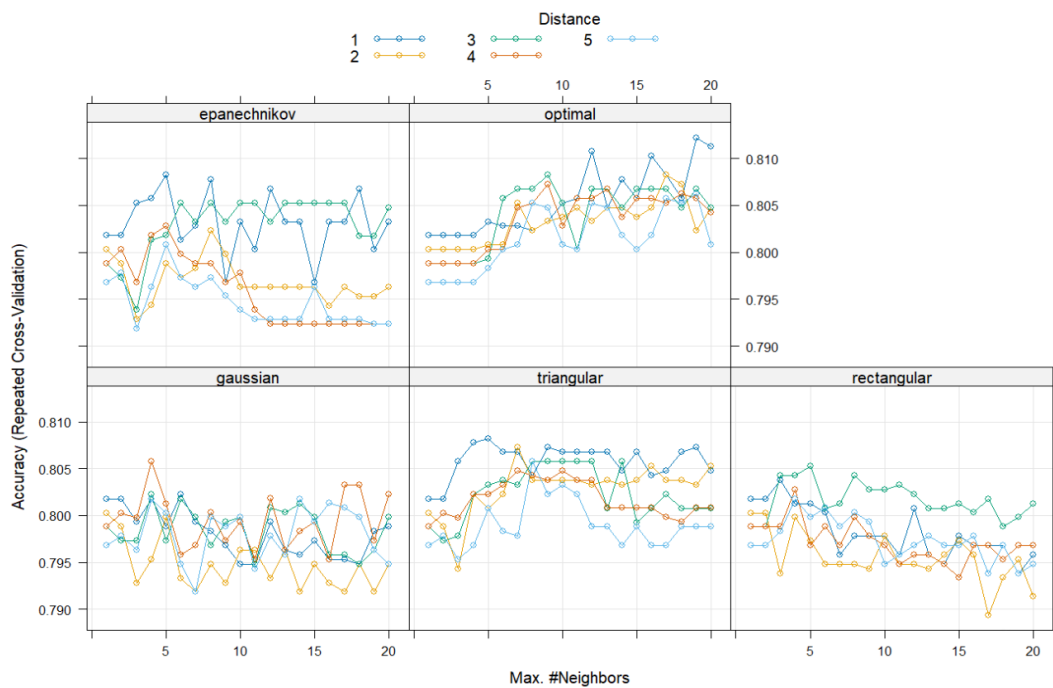


Figure 3. Model 3 - Accuracy of Different Kernel Functions Across Various Neighbors and Distances

3.3.2.2 Visual Analysis

The graphical representation of accuracy against the number of neighbors (k) for different kernel functions and distance metrics further illustrates the model performance. The second model consistently achieves higher accuracy across various configurations, highlighting the significance of balanced datasets and targeted duplicate removal in the "no" dataset.

The findings underscore the critical role of data preprocessing in enhancing the performance of KNN models. The second model, with its strategic approach to handling duplicates and balancing the dataset, outperforms the other models in terms of accuracy and kappa statistics. This suggests that preprocessing steps that specifically target dataset imbalances and redundancies can significantly boost classification performance.

Finally, the third model was constructed to satisfy the initial condition that if a class is true, it cannot be false. This model also identifies and removes duplicate entries in both the "yes" and "no" datasets. Although this model showed a lower performance in terms of accuracy (0.8018) and kappa (0.6036) compared to the other models, it was chosen for several reasons:

1. **Data Integrity:** The third model ensures a more rigorous approach to maintaining data integrity by removing duplicates from both classes, which is crucial for applications requiring high reliability and consistency.
2. **Class Consistency:** This model adheres strictly to the condition that a class cannot be true in one commit and false in another, providing a more robust classification framework.
3. **Comprehensive Preprocessing:** By addressing duplicates in both "yes" and "no" datasets, the model minimizes the risk of biased results due to redundant or conflicting data points.

In conclusion, while the second model displayed the highest performance metrics, the third model was selected for its thorough approach to data preprocessing and its alignment with the study's initial condition of class consistency. This decision underscores the importance of data quality and integrity in developing reliable and accurate machine learning models. Future research could explore additional preprocessing techniques and different machine learning algorithms to further enhance model performance and applicability.

This detailed and formal review provides a comprehensive analysis of the methodology, results, and rationale for choosing the third KNN model, contributing to a deeper understanding of effective data preprocessing techniques in machine learning.

3.3.3 Phase 2

Having examined the distinct methodologies and performance metrics of each model, a comprehensive approach was necessary to ensure the integrity and robustness of the dataset compositions and model configurations.

First Model: A balanced distribution of 'yes' and 'no' instances was achieved by allocating 70% of the total 'yes' instances and 70% of the total 'no' instances to the training set. Subsequently, the remaining 30% of both 'yes' and 'no' instances were designated for the test set. This methodology ensured equitable representation of both classes in both the training and test datasets.

Second Model: This model adopted a random selection approach, where an equivalent number of 'no' instances to 'yes' instances were randomly sampled for training and testing. Specifically, 70% of the combined instances were utilized for training, while the remaining 30% were allocated to testing.

Third Model: In this model, classes participating in the TRUE segment of the dataset were precluded from being randomly selected as FALSE from alternative commits. Additionally, duplicate entries were systematically eliminated to enhance dataset coherence.

3.3.3.1 Performance Results:

The results indicated that while the third model exhibited superior accuracy (0.9548) and the size of the training and test sets was 3402, the other models also performed commendably with accuracies of 0.9048 and 0.9145, respectively.

Table 1. Results of models in phase 2

	Accuracy	sizeTrain&Test
Model1	0.9048	980
Model2	0.9145	980
Model3	0.9548	3402

Ultimately, a judicious decision was made to segregate datasets from distinct projects for training and testing purposes. This approach ensured that both training and testing datasets adhered to the established criteria, including the prohibition of random selection of TRUE classes as FALSE instances and the preservation of a balanced distribution of 'yes' and 'no' instances in the training dataset. Overall, meticulous attention was devoted to the formulation of balanced dataset compositions and model configurations to facilitate robust model training and evaluation processes, thereby optimizing predictive performance outcomes.

Considering the rigorous preprocessing steps and adherence to the initial condition of class consistency, the third model was selected as the optimal configuration. This decision underscores the importance of data quality and integrity in developing reliable and accurate machine learning models.

3.4 Parameter Adjustment for KNN

Parameters play a pivotal role in configuring machine learning models, significantly influencing their predictive capabilities. Accordingly, a series of systematic experiments were undertaken to ascertain the most effective parameter combination, particularly within the framework of the k-Nearest Neighbors (KNN) algorithm.

To clarify the parameters within the kkn model, the following explanations are provided:

TuneGrid:

➤ kmax:

- ❖ Defines the number of nearest neighbors ($k = 1$ to 20) considered during the prediction process.
- ❖ Higher values of k lead to smoother predictions but may reduce precision.
- ❖ Lower values of k provide greater sensitivity to data fluctuations but may increase the risk of overfitting.

➤ **distance:**

- ❖ Dictates the distance metric used to compute the proximity between data instances.
- ❖ Common selections include:
 - **Euclidean distance:** Measures the straight-line distance between two points.
 - **Manhattan distance:** Computes distance by summing the absolute differences in coordinates.
 - **Minkowski distance:** Generalizes Euclidean and Manhattan distances.

➤ **kernel:**

- ❖ Specifies the weighting scheme applied to neighboring instances.
- ❖ The chosen kernel significantly impacts the influence exerted by each neighbor on the prediction outcome.
- ❖ Available kernel types include:
 - **Gaussian:** Gradual decrease in influence with increasing distance.
 - **Triangular:** Abrupt decline in influence after a certain threshold distance.
 - **Rectangular:** Uniform weighting to all neighbors.
 - **Epanechnikov:** Heavier weighting for closer neighbors, similar to Gaussian.
 - **Optimal:** Automatically adjusts the weighting mechanism to optimize predictive performance.

Optimal Selection of k:

- ❖ Utilization of `set.seed(56)` ensures reproducibility and consistency in results.
- ❖ The `kknn_fit` procedure facilitates model training based on the specified `tuneGrid` settings.
- ❖ The target variable `REFACTORED` is used alongside other predictor variables for model training.
- ❖ The dataset `data_btraining` serves as the foundational dataset for model training.
- ❖ Specification of `method = 'kknn'` denotes the adoption of the KNN algorithm for model training.

Additional Considerations:

- ❖ trControl governs various facets of the training process, including the methodology employed for cross-validation.
- ❖ The parameter tuneLength = 10 specifies the number of iterations for optimal parameter tuning.

Recommendations:

- ❖ Deliberate experimentation with diverse parameter values is advocated to pinpoint the optimal configuration tailored to the specific problem at hand.
- ❖ Employing visualizations and statistical analyses can help discern performance differences across various parameter configurations.
- ❖ Initial experimentation with a simplified parameter setting, followed by iterative refinement, is a prudent approach to parameter tuning.

Purpose:

- ❖ The objective of parameter tuning in the context of KNN is to identify the parameter combination that maximizes the model's predictive efficacy.
- ❖ This iterative process involves training KNN models for different parameter combinations delineated in tuneGrid, followed by performance evaluation based on pertinent metrics such as accuracy or Root Mean Squared Error (RMSE).
- ❖ The final selection entails choosing the parameter combination that yields optimal performance metrics.

```
# Tune kknn parameters
tuneGrid <- expand.grid(kmax = 1:20,
                      distance = 1:5,
                      kernel = c('gaussian', 'triangular', 'rectangular', 'epanechnikov', 'optimal'))
# Find the optimal k value
set.seed(56)
kknn_fit <- train(REFACTORED ~ .,
                 data = data_btraining,
                 method = 'kknn',
                 trControl = trctrl,
                 tuneGrid = tuneGrid,
                 tuneLength = 10)
```

3.5 Parameter Evaluation for Optimal Parameters

In the context of diverse datasets, the identification of optimal parameters varied, prompting the establishment of a rigorous ranking system to determine the most favorable parameter combination across the spectrum of datasets.

To adhere to the evaluation protocol, a comprehensive scoring mechanism was devised. Specifically, the top three parameter combinations received a maximum score of three points each, while the subsequent fourth and fifth combinations garnered two points each. The remaining combinations were allocated one point each.

The parameter combination accruing the highest cumulative score was conclusively designated as the optimal configuration. This combination was deemed the best parameter ensemble for the model training process

3.6 Detailed Description of Scripts for Predictive Analysis and Explanations

As a general practice, this process required a significant investment of time and resources. Consequently, it was collectively decided to establish a "best parameters" model, where the programmer would execute it initially and at predetermined intervals. Following this decision, scripts were meticulously crafted to facilitate various aspects of the predictive analysis process.

Script 1: Model Training and Prediction

This script facilitates predictive analysis by training a K-Nearest Neighbors (KNN) model and using it for predictions. It performs the following steps:

Input Handling:

- ❖ Begins by taking a CSV file as input, which contains the data for prediction.
- ❖ Requires a pre-trained KNN model to be loaded for making predictions.
- ❖ Requires an explainer, typically generated by a Lime script, to output the prediction probabilities.
- ❖ Accepts a name parameter for organizational purposes, used for naming the output file.

Data Processing:

- ❖ Reads the CSV file and prepares the data for prediction.
- ❖ Applies the model to the data to generate predictions.

Output Generation:

- ❖ Generates a JSON file containing the prediction results.
- ❖ Includes prediction probabilities in the JSON file, crucial for understanding the confidence of the model's predictions.

Script 2: Neighbor Information Retrieval

This script is crucial for retaining and analyzing information about neighbors in the dataset, particularly useful for understanding model behavior and predictions.

Input Handling:

- ❖ Accepts a specific class as input, which determines the target class for which neighbors will be retrieved.
- ❖ Requires the same CSV file used for training the KNN model to ensure consistency.

Neighbor Identification:

- ❖ Identifies and retrieves the neighbors of the specified class using the pre-trained KNN model.
- ❖ Plots the neighbors and visualizes their similarities to the target class.

Output Generation:

- ❖ Saves the neighbor information, including similarities to the target class, in a JSON file.
- ❖ Generates a 3D plot to visualize the neighbors and their relationships to the target class, saving this plot as an HTML file.

Script 3: Lime-Based Explanations

This script utilizes the Lime library to provide explanations for the predictions made by the KNN model. It helps in understanding which features contributed to a particular prediction.

Input Handling:

- ❖ Takes a CSV file as input for model training, typically the same file used to train the KNN model.
- ❖ Requires the specific instance we wish to explain.
- ❖ Loads the pre-trained KNN model and an explainer object created by Lime.
- ❖ Accepts an organizational variable that determines the output filename.

Explanation Generation:

- ❖ Applies the Lime explainer to the specified instance to generate an explanation.
- ❖ Identifies and highlights the key features that influenced the model's prediction.

Output Generation:

- ❖ Generates a JSON file containing the explanation results, including probabilities and feature contributions.
- ❖ Creates a visual representation of the explanation, saved as a PNG file.

These scripts collectively enable a robust framework for predictive analysis, neighbor information retrieval, and detailed explanations of model predictions. They ensure a systematic approach to model training, prediction, and interpretation, enhancing the transparency and reliability of the predictive modeling process. Each script is designed to handle specific aspects of the analysis, contributing to a comprehensive and interpretable machine learning pipeline.

3.7 Python

Transitioning from R to Python for implementing our scripts brings numerous benefits that positively impact the overall process of data development and analysis. This decision leverages the extensive and vibrant community of developers who continuously create new libraries and tools for data processing and analysis. Python offers a flexible and robust platform for machine learning development, supported by extensive libraries such as scikit-learn and TensorFlow. Additionally, adopting Python aligns with the broader trend towards its adoption in data science and machine learning. This facilitates collaboration and integration of data analyses with other tools and systems using the Python programming language. Thus, our transition from R to Python represents a natural and progressive step that enhances the effectiveness and flexibility of our analyses.

First Script - Parameters

As for Python, a script was done to have the K-Nearest Neighbors (KNN) model automatically select parameters for us. The script defines a range of values for k (number of neighbors), distance metrics, and weighting and then goes through the training of the model with each parameter combination and evaluation. The script will compare the F1 scores for that parameter combination with the best predictive performance. Using Python gives an assurance of broad machine learning and data analysis libraries in the process. At the same time, it provides the means to effectively and reproducibly tune parameters in search of improvements in the accuracy and reliability of the model.

```
# Define the range of values for k, distance, and weights
k_values = list(range(1, min(len(X_train), len(X_test), 21)))
distance_metrics = [1, 2, 3, 4, 5] # 1 for Manhattan distance, 2 for Euclidean distance
weights_options = ['distance']

best_f1 = 0
best_params = {}

# Iterate over parameter combinations
for k_value in k_values:
    for distance_metric in distance_metrics:
        for weights_option in weights_options:
            # Train k-NN model only if k_value <= number of samples in test set
            if k_value <= len(X_test):
                # Train k-NN model
                knn_model = KNeighborsClassifier(n_neighbors=k_value,
p=distance_metric, weights=weights_option)
                knn_model.fit(X_train, y_train)

                # Make predictions on test data
                y_pred = knn_model.predict(X_test)
                accuracy = accuracy_score(y_test, y_pred)
                f1 = f1_score(y_test, y_pred, average='weighted')
```

```

        # Print or store the results
        print(f'k={k_value}, distance={distance_metric}, weights={weights_option}:
f1={f1:.2f}')
        # Update the best parameters if the current model has higher accuracy
        if f1 > best_f1:
            best_f1 = f1
            best_params = {'k': k_value, 'distance': distance_metric, 'weights':
weights_option}

```

Second Script - K-NN Classification: This script focuses on implementing the K-Nearest Neighbors (K-NN) algorithm for classification tasks. K-NN is a straightforward yet effective machine learning algorithm used for classification and regression tasks. By rewriting the R script in Python, we ensure compatibility with other Python-based tools and libraries commonly used in data science and machine learning workflows. Python offers a rich ecosystem of libraries like scikit-learn for machine learning tasks, making it an ideal choice for implementing algorithms like K-NN. The script handles tasks such as data preprocessing, model training, evaluation using metrics like accuracy and F1-score, and visualization of results using libraries like seaborn and matplotlib. By leveraging argparse, the script allows for easy command-line interaction, making it convenient to specify parameters like file paths, k-value, and distance metric from the command line.

```

X_train = data_training_processed.drop('REFACTORED', axis=1)
y_train = data_training_processed['REFACTORED']
X_test = data_test_processed.drop('REFACTORED', axis=1)
y_test = data_test_processed['REFACTORED']

knn_model = KNeighborsClassifier(n_neighbors=k_value, p=distance_metric,
weights='distance')
knn_model.fit(X_train.values, y_train.values)

y_pred = knn_model.predict(X_test.values)

```

Thirs Script - Prediction with K-NN: This script focuses on making predictions using a trained K-NN model. It takes as input a CSV file containing the data to be predicted and outputs the predictions in a JSON format. By using argparse, the script facilitates easy specification of input and output file paths, as well as the path to the pre-trained K-NN model. The use of pandas for data manipulation and pickle for model serialization ensures compatibility and ease of use. This script streamlines the prediction process, making it efficient and straightforward.

```
y_pred = knn_model.predict(X_test.values)
y_proba = knn_model.predict_proba(X_test.values)
data_test = data_test.drop('REFACTORED', axis=1)
results_df = pd.concat([pd.DataFrame(data_test), pd.Series(y_pred,
name='Predicted'), pd.DataFrame(y_proba, columns=['Probability_0', 'Probability_1'])],
axis=1)
result_json = results_df[['file', 'Predicted', 'Probability_0',
'Probability_1']].to_json(orient='records', lines=True)
```

Fourth Script - Data Visualization and Analysis: In this script, the emphasis is on visualizing and analyzing data. It leverages libraries like plotly.graph_objects and seaborn for creating interactive and insightful visualizations. The script reads data from CSV files, performs necessary preprocessing steps, and generates visualizations such as scatter plots to gain insights into the data distribution and relationships between variables. By incorporating argparse, the script allows for easy specification of input and output file paths, as well as parameters for customizing the visualization.

```
# select specific row
selected_row_indices = np.where(data['file'] == selected)[0]
datab = data.iloc[:, 3:]

# select specific rowReturns neighbors along with distances
distances, neighbors = knn_model.kneighbors(datab.iloc[selected_row_indices,
:-1].values,

return_distance=True)

sorted_indices = np.argsort(distances)
```

```

neighbors_sorted = neighbors[:, sorted_indices]
distances_sorted = distances[:, sorted_indices]
neighbors_sorted = neighbors_sorted[0]
distances_sorted = distances_sorted[0]
# Adding the neighbors to the DataFrame
result_dfa = pd.DataFrame()
for i in range(neighbors_sorted.shape[1]):
    test_data = data_training.iloc[neighbors_sorted[:, i], :].copy()
    test_data.loc[:, 'Distance'] = distances_sorted[:, i]
    result_dfa = pd.concat([result_dfa, test_data])

```

```

# Creating the scatter plot for "yes"
yes_trace    =    go.Scatter3d(x=result_df[result_df['REFACTORED'] ==
1][param1],
                                y=result_df[result_df['REFACTORED'] == 1][param2],
                                z=result_df[result_df['REFACTORED'] == 1][param3],
                                mode='markers',
                                marker=dict(color="blue", size=10, opacity=1),
                                name="Yes")

# Creating the scatter plot for "no"
no_trace     =    go.Scatter3d(x=result_df[result_df['REFACTORED'] ==
0][param1],
                                y=result_df[result_df['REFACTORED'] == 0][param2],
                                z=result_df[result_df['REFACTORED'] == 0][param3],
                                mode='markers',
                                marker=dict(color="black", size=10, opacity=1),
                                name="No")

# Creating the scatter plot for specific spot
selected_trace = go.Scatter3d(x=data.loc[selected_row_indices][param1],

```



```

        y=data.loc[selected_row_indices][param2],
        z=data.loc[selected_row_indices][param3],
        mode='markers',
        marker=dict(color="red", size=15, opacity=1),
        name="Selected Class")

# Construction of the overall diagram
fig = go.Figure(data=[yes_trace, no_trace, selected_trace])

fig.update_layout(scene=dict(xaxis_title=param1,          yaxis_title=param2,
zaxis_title=param3))

name = output_file + "_plot.html"
# Save plot as HTML
fig.write_html(name)

```

Fifth Script - Explaining K-NN Classifications with LIME: This script integrates LIME (Local Interpretable Model-agnostic Explanations) to provide explanations for K-NN classifications. LIME is a popular technique for explaining the predictions of machine learning models by approximating their behavior locally. By incorporating LIME into the script, we gain insights into how the K-NN model makes predictions for individual instances. The script reads the pre-trained K-NN model and the data to be explained, applies LIME to generate explanations, and saves the explanations in a visual format. This script enhances the interpretability of the K-NN model, making it easier to understand and trust its predictions.

By rewriting the R scripts in Python and providing detailed explanations of each script's functionality, we ensure consistency and clarity in the implementation and understanding of the machine learning workflow. This approach enables seamless integration with existing Python-based tools and libraries, fostering collaboration and productivity in data science projects.

```

with open(knn_model, 'rb') as model_file:
    knn_model = pickle.load(model_file)

data_training = data_training.iloc[:, 3:-1] # Αφαίρεση των πρώτων 4 στηλών

feature_names = list(data_training.columns) # Υποθέτοντας ότι η τελευταία στήλη
είναι η μεταβλητή-στόχος

explainer = lime_tabular.LimeTabularExplainer(data_training.values,
feature_names=feature_names, class_names=[0, 1], discretize_continuous=True)
knn_model.feature_names = feature_names
selected_row = data[data['file'] == selected]
selected_row_for_explanation = selected_row.iloc[:, 3:-1]
explanation = explainer.explain_instance(selected_row_for_explanation.values[0],
knn_model.predict_proba)
print("explanation fit: "+explanation.score)

fig = explanation.as_pyplot_figure()
fig.set_size_inches(10, 6) # Set the figure size explicitly
plotname = output_file + '_plot.png'
fig.savefig(plotname, bbox_inches='tight') # Use bbox_inches='tight' to avoid cutting
off

```

3.8 Validation with Python

With respect to the project, it can be further enhanced and improved by: optimizing and restructuring code for clarity, applying a more efficient way to deal with imbalanced data, and incorporating cross-validation and hyperparameter tuning. Some improvements have already been made, such as using `argparse` to handle command-line arguments, better handling of imbalanced data, and implementing cross-validation and hyperparameter tuning. The next paragraphs indicate the specific changes done.

The transition from the first version of code toward the current improved version is a giant leap in flexibility, efficiency, and reliability. Improvements do not only focus on the optimization of the current workflow but also on laying solid foundations for further advancement.

Through argument handling, data balancing, hyperparameter tuning, and performance evaluation, a vital area of the project is covered, which enables better treatment of more complex and variable datasets and provides more reliable results.

3.8.1 Comparative Analysis and Benefits of New Approach

3.8.1.1 `argparse` for Managing the Command Line

The previous code thus achieved handling of command-line arguments with the help of the ``argparse`` package. This helped in specifying paths to train and test CSV files and obtaining the name of the output JSON file from the user. This made it flexible, but users had to change the code every time for a different file path. The following updated way uses ``argparse`` the same way—just in a bit more streamlined and flexible manner. Now, a user can specify the file path and an output file directly from the command line without changing the script. So now, the code is more user-friendly and adaptable to different usage scenarios. The new ``argparse`` way makes this especially simple in specifying input and output files through command-line arguments. The code is very flexible for various scenarios.

3.8.1.2 Imbalanced Data Handling

In the previous code, imbalanced data was handled by manual undersampling of the majority class. Mainly, it involved the selection of random subsets of indices in the majority and minority classes to come up with balanced data. Now, the approach automatically performs under-sampling using `RandomUnderSampler` from the `imbalanced-learn` library. That way, the dataset will always be balanced and gotten more automatically and reliably. The new method under the case of undersampling ensures a balanced dataset. It is carried out using `imbalanced-learn`, for which the implementation is far more efficient and reliable, enhancing better performance in models while reducing bias towards the majority class.

3.8.1.3 Implementation of Cross Validation and Tuning of Hyperparameters

The previous technique iteratively checked different combinations of hyperparameters: number of neighbors, distance metric, and weights in the k-NN model, based on which it picked the best combination considering the F1 score. This search process is very unsystematic and not cross-validated at all. Our implementation included using `GridSearchCV` for systematic tuning of hyperparameters and `KFold` cross-validation to evaluate the model on several data subsets. Hence, it gives a more complete and robust result. Using `GridSearchCV` enables one to systematically and thoroughly tune the hyperparameters so that the model offers the best possible performance. Employing `KFold` cross-validation gives a much better estimate of model performance because you evaluate it across several data splits and decrease, thereby reducing the chance of overfitting.

3.8.1.4 Result Calculation and Storage

In the past code, the predictions were made on the test set using metrics such as accuracy and F1 score after calculation with the best parameters and scores stored in a JSON file.

The corrected cross-validation makes calculations of the key metrics from each fold: accuracy, F1 score, precision, recall, and ROC-AUC. On top of that, save the best model with its tuned hyperparameters using `pickle`, so one can easily reuse it without retraining for more time/resource savings. Hence, it gives more reliable and repeatable results compared to a simple train/test split. Saving the best model and tuning its hyperparameters using `pickle` allows easy reuse without retraining, saving a lot of time and resources.

The new approach, which has improved in terms of usability, accuracy, and flexibility, is far from the earlier one. `argparse` manages command line arguments; `imbalanced-learn` recompenses an imbalance of data; `GridSearchCV` tunes the hyperparameters; and `KFold` cross-validates the performance evaluation for robust, reliable, and user-friendly training and evaluation of models. All these enhancements increase adaptability and efficiencies, ensuring a more excellent foundation upon which future development and deployment may safely be constructed.

3.8.2 Code

```
knn_model = KNeighborsClassifier()

kf = KFold(n_splits=10, shuffle=True, random_state=42)

beta = 2
f2_scorer = make_scorer(fbeta_score, beta=beta)

param_grid = {
    'n_neighbors': list(range(1, 21)), # Παράδειγμα τιμών για τον αριθμό των γειτόνων
    'p': [1, 2], # Παράδειγμα τιμών για το p στον τύπο Minkowski
    'weights': ['distance'] # Παράδειγμα τιμών για το βάρος των γειτόνων
}

grid_search = GridSearchCV(knn_model, param_grid, scoring=f2_scorer)
grid_search.fit(X, y)

print("Best Parameters:", grid_search.best_params_)
print("Best F2 Score:", grid_search.best_score_)

for i, (train_index, test_index) in enumerate(kf.split(X)):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    grid_search.best_estimator_.fit(X_train, y_train)

    y_pred = grid_search.best_estimator_.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='macro')
    f2 = fbeta_score(y_test, y_pred, beta=beta)

    print(f"Fold {i + 1} - Accuracy: {accuracy}, F1 Score: {f1}, F2 Score: {f2}")
```

```
f1_scores = cross_val_score(knn_model, X, y, cv=kf, scoring='f1_macro')
precision_scores = cross_val_score(grid_search.best_estimator_, X, y, cv=kf,
scoring='precision')
recall_scores = cross_val_score(grid_search.best_estimator_, X, y, cv=kf,
scoring='recall')
roc_auc_scores = cross_val_score(grid_search.best_estimator_, X, y, cv=kf,
scoring='roc_auc')
f2_scores = cross_val_score(knn_model, X, y, cv=kf, scoring=f2_scorer)
```

4 Integration with debt clock platform

Although the performance of the newer model with cross-validation is better, the Python models were first used in the application. This will allow it to be consistent with the flow that has been set about compatibility and stability with the previous state so that improvements can be introduced gradually.

4.1 For the Application

It is important to acknowledge that the designated projects originate from the Apache Foundation. The primary interface presents users with an overview of the company's projects, along with relevant project details. Specifically, users can access information such as the total number of selected projects, the lines of code these projects encompass, and the extent of refactoring efforts within the organization. Additionally, a graphical representation allows users to visualize the additional debt and cumulative debt incurred on a weekly basis throughout the organization's history.

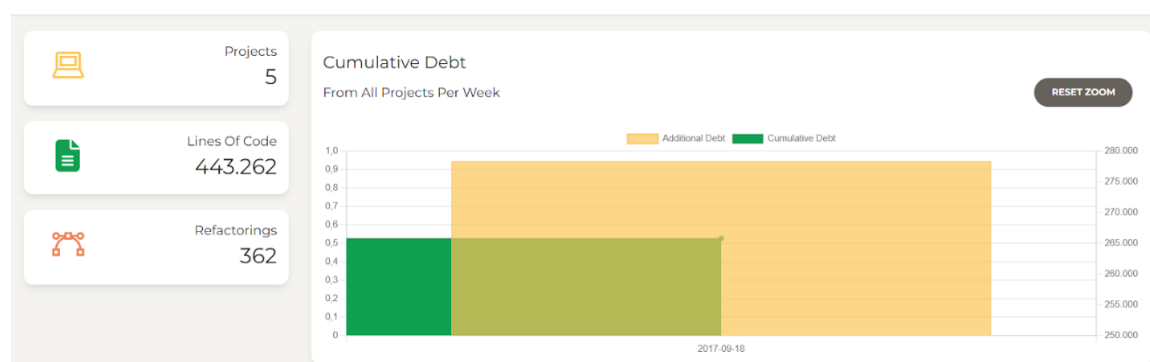


Figure 4. Dashboard of application

Below this overview, the interface highlights the top 5 contributors with the highest cleanliness ratio and those who perform the most refactorings. The term "cleanliness" refers to the proportion of technical debt accrued relative to the lines of code authored by each contributor. For instance, an individual who accumulates \$5 of technical debt for every 100 lines of code they produce would be deemed "cleaner" than someone who amasses \$10 of technical debt for the same amount of code. In both scenarios, the top 5 contributors are prominently featured.

Contributors actively engaged in refactoring endeavors are showcased on the right, while those with the most exemplary cleanliness ratios are highlighted on the left.

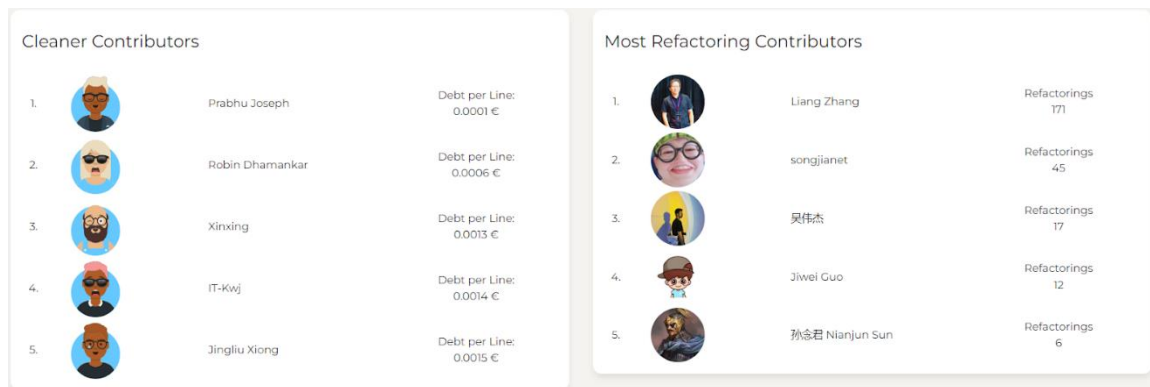


Figure 5. Top Contributors in Code Quality and Refactoring

For each project, users have four options available. The first option allows access to detailed information about the specific project. The second option relates to DeepInCode, while the third pertains to CodeWise.

First Option: Detailed Project Information

- ❖ Users can review specific metrics such as Cumulative Debt, Weekly Debt, the Interest Rate (both pertinent to technical debt), the lines of code, and the number of refactorings performed within the project.
- ❖ On the right-hand side, a diagram illustrates these metrics up to the latest commit analyzed, providing insights into Cumulative Debt, Weekly Debt, and Interest Rate on a weekly basis.
- ❖ Further below, users can access average project metrics and metrics broken down by file.
- ❖ The lower section of the same screen displays the top contributors with the highest cleanliness ratios and those who execute the most refactorings specifically for the selected project.

Second Option: DeepInCode

- ❖ Presents a heatmap on the interface, depicting the heightened probability of technical debt per class.
- ❖ This probability is derived from an artificial intelligence (AI) model specifically developed for this purpose.
- ❖ The model was crafted using tools designed to identify technical debt, including SonarQube, across a corpus of 100 projects.

- ❖ The model evaluated the correlation of decisions made by each tool and their significance to the model. For instance, if a tool flagged a file with significant technical debt while others did not, the model would give it less weight. Conversely, if all tools agreed on high technical debt, the model would scrutinize it more closely.
- ❖ Consequently, the model has been trained to discern classes presenting a heightened risk of technical debt.

These options provide users with comprehensive insights and tools for managing technical debt and understanding the codebase's quality and maintenance needs.

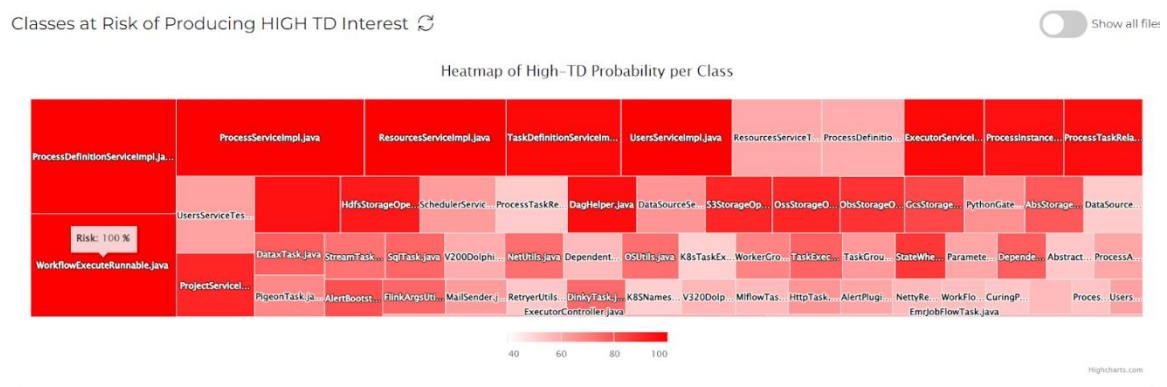


Figure 6. Heatmap of High – TD Probability per Class

Beneath this section, the SHAP (SHapley Additive exPlanations) technique is utilized, focusing on the Root Cause Analysis of High Technical Debt (TD) Interest Risk. Two diagrams are displayed: one outlining the root causes and the other showing the metrics threshold. When a file is selected, a force plot diagram, commonly known as a SHAP force plot, is generated. This plot visually represents the current value and the influence exerted by various factors, based on the chosen value.

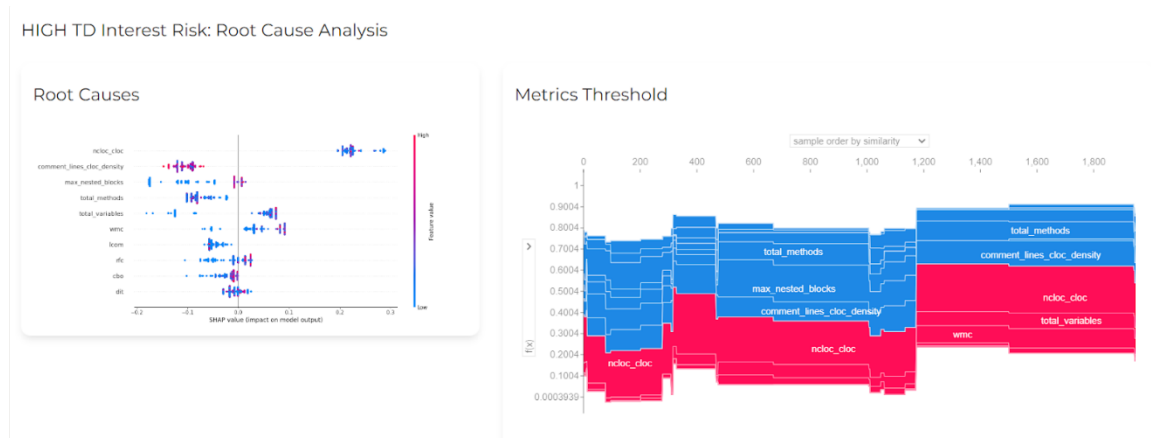


Figure 7. SHAP – HIGH TD interest Risk

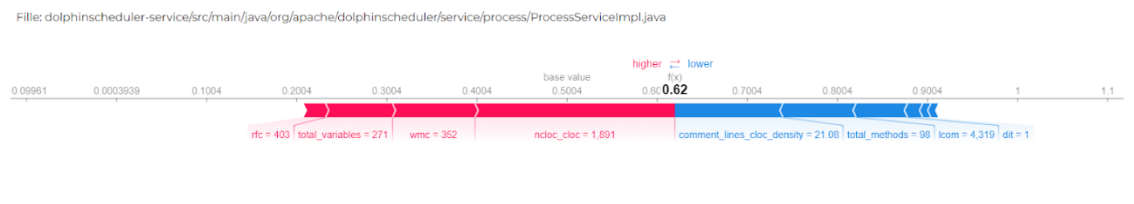


Figure 8. SHAP - force plot

4.1.1 Back end

Java Spring is widely recognized as a leading open-source framework for developing Java enterprise applications. It embodies the principles of inversion of control and loose coupling, resulting in codebases that are adaptable, scalable, and easy to maintain (Johnson, R., et al., 2004).

At its core, Java Spring emphasizes the inversion of control principle, where the framework manages the flow of application execution. This paradigm shift frees developers from managing control flow, providing greater flexibility and extensibility in application architecture (Fowler, M., 2004).

Java Spring also promotes loose coupling between components, reducing interdependencies and enhancing modularity within the codebase. By decoupling components, Java Spring allows for the creation of software solutions that are resilient to change and easily integrate with evolving technologies (Walls, C., 2011)

In essence, Java Spring empowers developers to create resilient, adaptable software solutions suited for modern enterprise environments.

Key Features:

➤ **Model-View-Controller (MVC) Architecture:**

- ❖ Separates an application into three components: Model (data management), View (presentation layer), and Controller (application logic) (Fowler, M., 2003)

➤ **Dependency Injection:**

- ❖ Facilitates the management of dependencies, enabling seamless unit testing and promoting modularity by decoupling components (Johnson, R., et al., 2004).

- **Aspect-Oriented Programming (AOP):**
 - ❖ Allows developers to modularize cross-cutting concerns (e.g., logging, security) separately from the application's core logic (Kiczales, G., et al., 1997).
- **Spring Boot:**
 - ❖ Adopts a convention-over-configuration approach, simplifying the development of microservices and web applications by minimizing manual configuration (Walls, C., 2016)
- **Spring Data:**
 - ❖ Provides consistent data access across various data sources, including SQL databases, NoSQL databases, and cloud-based data stores (Long, J., 2017).
- **Spring Security:**
 - ❖ A robust framework for securing web applications, offering comprehensive authentication, authorization, and other security features (Deinum, M., et al., 2018).

Applications:

- **Web Applications:**
 - ❖ Java Spring supports the development of RESTful APIs, single-page applications (SPAs), and web portals, creating robust and scalable web-based solutions (Walls, C., 2016)
- **Enterprise Applications:**
 - ❖ Plays a crucial role in building complex enterprise systems like ERP platforms, CRM solutions, and BI systems, providing a framework for managing and analyzing business data effectively (Johnson, R., et al., 2004).
- **Cloud Applications:**
 - ❖ Supports microservices architectures, cloud-native applications, and serverless computing, facilitating the development of scalable and resilient cloud-based solutions (Long, J., 2017)

4.1.1.1 Spring Boot:

Spring Boot, an open-source Java framework, is designed for developing microservices. Its primary function is to facilitate the creation of standalone, production-grade Spring-based applications with minimal effort. It serves as a convention-over-configuration extension for the Spring Java platform, simplifying configuration complexities while ensuring seamless creation of Spring-based applications (Walls, C., 2016).

Spring Boot follows an "opinionated view" approach, advocating best practices for configuring and using the Spring platform and third-party libraries. Its preconfigured settings streamline the development process, reducing the burden of manual configuration for developers (Long, J., 2017)

Spring Boot's ease of use and robustness make it popular for various software projects, from microservices to web applications and other Java-based endeavors. Its widespread adoption highlights its effectiveness in simplifying development tasks and enhancing the reliability of software solutions (Walls, C., 2016).

Components:

Controllers:

➤ Responsibilities:

- ❖ Handle incoming HTTP requests from users.
- ❖ Process requests and generate appropriate responses, including data retrieval and view rendering.
- ❖ Control the application flow by orchestrating interactions between various components.

➤ Characteristics:

- ❖ Annotated with `@Controller` to denote their role within the application architecture.
- ❖ Use annotations like `@RequestMapping` to map specific URLs to corresponding methods.
- ❖ Interact with services to implement the required business logic and fulfill user requests.

Services:

➤ Responsibilities:

- ❖ Provide encapsulated business logic and high-level services, abstracting away implementation details.

- ❖ Act as an intermediary layer between controllers and repositories, facilitating the execution of complex operations.
- ❖ Communicate with repositories to access and manipulate data stored in the underlying database.

➤ **Characteristics:**

- ❖ Annotated with `@Service` to signify their role as service components within the application context.
- ❖ Do not directly handle HTTP requests or responses but offer reusable methods invoked by controllers or other services.
- ❖ Focus on implementing domain-specific functionality and enforcing business rules.

Repositories:

Responsibilities:

- ❖ Manage the persistence and retrieval of data from the underlying data store, such as a database.
- ❖ Implement data access logic following established patterns like DAO (Data Access Object).
- ❖ Shield the higher layers of the application, such as services, from the intricacies of data access and storage.

Characteristics:

- ❖ Typically annotated with `@Repository` to indicate their role as data access components within the application context.
- ❖ Leverage frameworks like Spring Data JPA to streamline database access and simplify CRUD (Create, Read, Update, Delete) operations.
- ❖ Offer methods for interacting with database entities while abstracting away low-level database operations.

Collaboration:

Controller:

- ❖ Functions as the front-facing component, responsible for receiving and processing HTTP requests from clients.
- ❖ Upon receipt of a request, the controller delegates the execution of the requisite business logic to the service layer.

- ❖ Manages the presentation of data or views to the client, acting as the intermediary between the user interface and the underlying application logic.

Service:

- ❖ Positioned as the intermediary between the controller and the data access layer, the service layer orchestrates the application's business logic.
- ❖ Uses repositories for data access, retrieving, processing, and manipulating data as required by the application's functionality.
- ❖ Once processing is complete, the service layer provides the controller with the processed data or results for further dissemination to clients.
- ❖ This architectural abstraction ensures a modular and maintainable separation of concerns within the application.

Repository:

- ❖ Operates as the data access component, interfacing directly with the underlying data store, typically a database.
- ❖ Executes CRUD operations on data entities.
- ❖ Encapsulates database interactions, shielding the service layer from the intricacies of data access, promoting a clean separation of concerns.
- ❖ Entities or collections of entities retrieved from the repository are subsequently passed to the service layer for further processing, facilitating efficient data management within the application architecture.

Advantages:

➤ **Decoupling of Concerns:**

- ❖ The architectural design facilitates a clear segregation of responsibilities among distinct components, leading to a more adaptable and manageable codebase (Fowler, M., 2003)
- ❖ By delineating specific tasks to each component—such as controllers, services, and repositories—the system achieves a precise separation of (Johnson, R., et al., 2004).
- ❖ This architectural approach enhances the comprehensibility, maintainability, and scalability of the codebase, as each component focuses exclusively on its designated responsibilities concerns (Walls, C., 2016).

➤ **Enhancing Reusability:**

- ❖ Within the application architecture, services are engineered to be reusable across multiple controllers, thereby mitigating code redundancy and fostering code efficiency (Long, J., 2017).
- ❖ By encapsulating common business logic and functionalities within services, developers can invoke these services from various controllers without the need for duplicative code implementation concerns (Walls, C., 2016).
- ❖ This practice promotes code maintainability and fosters a modular design paradigm, facilitating smoother application development and evolution (Fowler, M., 2003)

➤ **Facilitating Testability:**

- ❖ The modular organization of the application, characterized by clearly defined layers and components, facilitates independent testing of each constituent part (Fowler, M., 2003)
- ❖ This granular approach enables developers to isolate and test individual components—such as controllers, services, and repositories—in isolation. concerns (Johnson, R., et al., 2004).
- ❖ Consequently, comprehensive testing scenarios can be devised to verify the functionality and behavior of each component independently, resulting in heightened code quality, reliability, and streamlined debugging processes. concerns (Walls, C., 2016).

4.1.1.2 Back-End Components

To comprehensively describe the back-end components of the Java Spring Boot application, we will analyze two key packages: `gr.nikos.dashboard.services.urgent` and `gr.nikos.dashboard.controllers`. These packages contain essential classes and functionalities that facilitate the automation of code refactoring analysis and prediction tasks. Let's delve into each package and its constituent classes:

4.1.2.1.1 Package: *gr.nikos.dashboard.controllers*

Class: UrgentPyController

This REST controller class defines endpoints for client-server communication related to code refactoring analysis and prediction. It exposes functionalities to trigger model training, retrieve project-wide and file-specific refactoring predictions, and obtain visualization data for analysis.

Key Endpoints:

- ❖ `/train`: Initiates the training of machine learning models for code refactoring prediction based on project data.
- ❖ `/project/results`: Retrieves project-wide refactoring predictions for further analysis.
- ❖ `/file/results/more`: Calculates detailed refactoring metrics for a specific file within the project.
- ❖ `/file/neighborhood/calculate`: Computes the neighborhood of a given file within the project based on similarity metrics.

Class: UrgentPyController

This REST controller class acts as a bridge between the front-end client and the back-end services for code refactoring analysis. It defines endpoints to trigger model training, retrieve prediction results, and obtain visualization data for presentation to the user.

Key Endpoints:

- ❖ `/train`: Handles requests to initiate the training of machine learning models for code refactoring prediction.
- ❖ `/project/results`: Retrieves project-wide refactoring predictions for visualization and analysis.
- ❖ `/file/results/more`: Calculates detailed refactoring metrics for a specific file within the project.
- ❖ `/file/neighborhood/calculate`: Computes the neighborhood of a given file within the project based on similarity metrics.

4.1.2.1.2 Package: *gr.nikos.dashboard.services.urgent*

Class: UrgentPyService

This service class orchestrates the training of machine learning models and the generation of code refactoring predictions. Its primary functionalities include:

- ❖ **Model Training:** Uses provided project data to generate CSV files containing commits, files, metrics, and refactorings. These files are then used to train machine learning models using Python scripts.
- ❖ **Prediction Generation:** Calculates project-wide and file-specific refactoring predictions based on trained models. Utilizes Python scripts to generate prediction outputs and extracts relevant information for further processing.
- ❖ **Error Handling:** Implements robust error handling mechanisms to manage exceptions encountered during the execution of Python scripts or file operations.

Key Methods:

- ❖ `trainModel(List<Project> projects, Project project)`: Handles the training of machine learning models by generating CSV files and executing Python scripts to train the models.
- ❖ `calculateProjectResults(Project project)`: Computes project-wide refactoring predictions and retrieves prediction results for further analysis.
- ❖ `calculateFileResults(Project project, String selectedFile)`: Calculates refactoring metrics for a specific file within the project and generates prediction results.
- ❖ `calculateFileNeighborhood(Project project, String file)`: Determines the neighborhood of a given file within the project based on similarity metrics.

These back-end components are crucial for automating code refactoring analysis and prediction tasks within the Java Spring Boot application. By integrating machine learning models and Python scripts, orchestrating model training, and providing RESTful endpoints for client-server communication, these components form the backbone of a robust and scalable solution for enhancing code quality and maintainability.

4.1.2 The model in the platform

If the user chooses the third option related to CodeWise, they will initially be presented with a heatmap visualizing the High Refactoring Probability per Class on the interface. Specifically, only classes with a probability exceeding 0.5 will be displayed on the diagram. However, the user can reveal all classes on the diagram by activating the "show all files" function. Classes highlighted in red indicate a refactoring probability of 1, while the color gradient towards white signifies classes with a refactoring probability of 0.

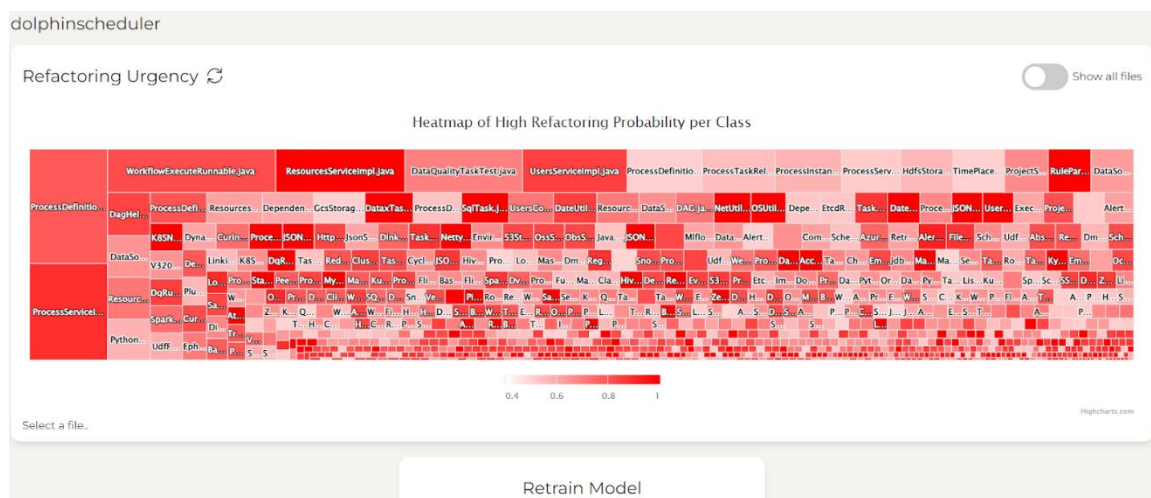


Figure 9. Heatmap of High Refactoring Probability per Class

Upon selecting a class from the heatmap, a subsequent Lime chart will appear below, illustrating the detailed influence of each metric on the decision that the class requires refactoring. This involves using weights from a simplified model for each feature to explain how each feature contributed to the prediction for that sample when using the original complex model. Positive contributions are typically shown as green lines, indicating an increase in the prediction due to higher feature values, while negative contributions are shown as red lines, signifying a decrease in the prediction due to lower feature values.

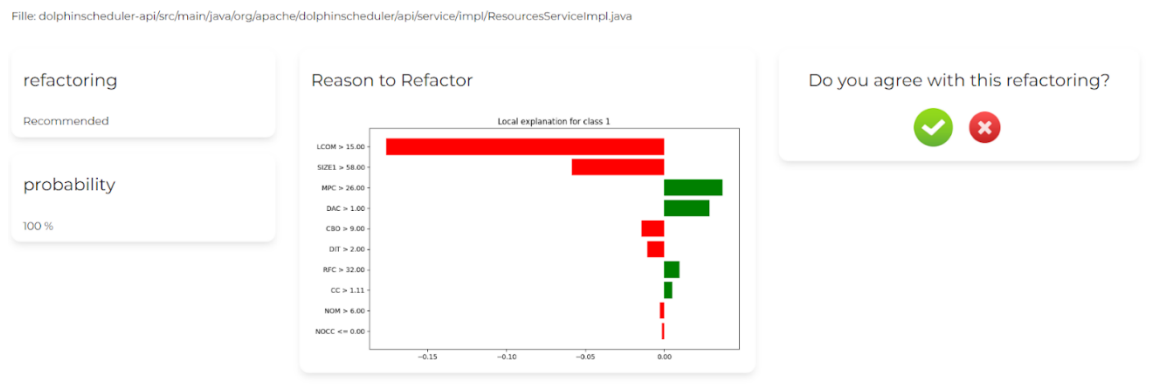


Figure 10. Lime plot

Following the user's selection, they are given the opportunity to either validate or contest the model's determination regarding the need for refactoring in the specific class. If the user disagrees, two figures are displayed on the interface.

Figure 11 on the left delineates the closest neighbors of the selected class, from which the class derived influence for the refactoring decision. Detailed information about these nearest neighbors is provided, including metric values such as LCOM, CC, and CBO, along with the similarity percentage. Additional pertinent details, including a "diff" button, are included. By activating this button, the user can examine a segment of the code and see the precise alterations made, facilitating their agreement or disagreement. The user can also specify neighbors with whom they disagree regarding the classification of changes as refactoring.

Figure 11 on the right presents a plot where the red dot denotes the selected class, and the blue dots denote its neighboring entities.

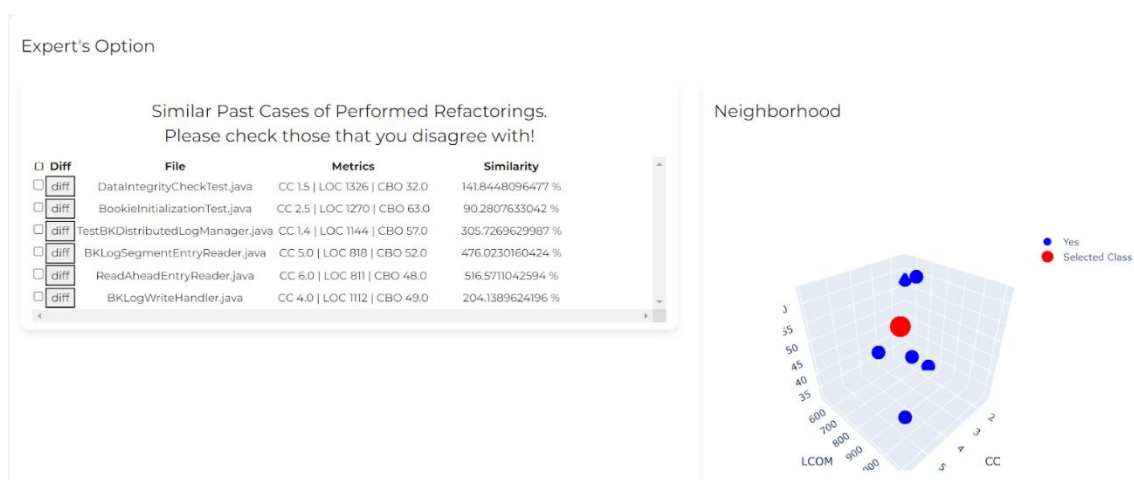


Figure 11. Similar Past Cases of Performed Refactorings

5 Results

5.1 Python Model - Using Data from All Projects Except the One to Be Predicted

To evaluate the performance of the prediction models, experiments were conducted on five different projects. Initially, for each project, five files were created where each file contained data from all projects except the one to be predicted. These files were split into training and testing sets with a 70/30% ratio. The script `parameters` and `knnmodel` was then executed, with the training set and test set consisting of data from all other projects. The results were as follows:

For the Bookkeeper project, the best parameters were $k=3$, $\text{distance}=1$, and $\text{weights}=\text{'distance'}$, resulting in a Best F1 score of 0.73. Similarly, for the Dolphinscheduler project, the best parameters were $k=2$, $\text{distance}=1$, and $\text{weights}=\text{'distance'}$, achieving a Best F1 score of 0.72. For the Hudi project, the best parameters were $k=2$, $\text{distance}=1$, and $\text{weights}=\text{'distance'}$, with a Best F1 score of 0.72. The Incubator-celeborn project performed best with $k=8$, $\text{distance}=1$, and $\text{weights}=\text{'distance'}$, attaining a Best F1 score of 0.70. Finally, for the ShardingSphere-elasticjob project, the best parameters were $k=12$, $\text{distance}=1$, and $\text{weights}=\text{'distance'}$, achieving the highest Best F1 score of 0.79.

Prediction and Neighbors Analysis

Subsequently, predictions were made for the project of interest. For a specific class, its neighbors were identified. However, these neighbors were from different projects, which prompted a change in approach. To address this issue, it was decided that the model would use a 70/30 split of the same project for both training and testing, with the prediction being made on the latest version of the specific project. This adjustment ensures that the neighbors displayed are classes from the same project rather than from different ones.

This decision was made to maintain the relevance and context of the neighboring classes, thereby enhancing the accuracy and reliability of the prediction and analysis process. By using the same project for training and testing, the model provides more meaningful insights and a more coherent analysis of the project's internal structure and relationships.

5.2 Python Model - Using Data from the Same Project

When applying the 70/30 split of the same project for training and testing, the results were as follows:

For the Bookkeeper project, the optimal parameters were $k=20$, $\text{distance}=1$, and $\text{weights}=\text{'distance'}$, resulting in a Best F1 score of 0.75. For the Dolphinscheduler project, the best parameters were $k=16$, $\text{distance}=5$, and $\text{weights}=\text{'distance'}$, achieving a Best F1 score of 0.84. In the case of the Hudi project, the optimal parameters were $k=19$, $\text{distance}=2$, and $\text{weights}=\text{'distance'}$, with a Best F1 score of 0.89. For the Incubator-celeborn project, the best parameters were $k=5$, $\text{distance}=3$, and $\text{weights}=\text{'distance'}$, achieving a Best F1 score of 0.88. Finally, for the Shardingsphere-elasticjob project, the optimal parameters were $k=10$, $\text{distance}=5$, and $\text{weights}=\text{'distance'}$, resulting in a Best F1 score of 0.72.

By using the same project for training and testing, this approach ensures the relevance and accuracy of the neighboring classes, enhancing the reliability of the prediction and analysis process.

5.3 Validation with Python Model - Results

Table 2. Results of Bookkeeper project

Fold	Accuracy	F1 Score	F2 Score
1	0.651	0.651	0.680
2	0.698	0.697	0.762
3	0.690	0.690	0.693
4	0.738	0.734	0.670
5	0.810	0.806	0.833
6	0.738	0.725	0.813
7	0.738	0.738	0.734
8	0.810	0.806	0.877
9	0.738	0.738	0.800
10	0.619	0.611	0.741

These evaluation results shed further light on its performance across key metrics. The dataset encompassed 1996 negative instances and 211 positive instances, hence providing a context for the model to handle imbalanced data. The best parameters were 11 neighbors, a power parameter equal to 2, and distance-based weights that generated an F2 Score of 0.7875. In our F1 score, the balance between precision, at 0.7023, and recall, at 0.7795, shows a reasonable figure of 0.6907. The reason the F2 score is much higher is that it puts more emphasis on recall, which aligns very well with the higher recall value presented by the model. What is more, the ROC-AUC score of the model, amounting to 0.7672, only highlights how effective this model is in distinguishing classes and, therefore, being pretty robust in carrying out classification tasks.

Table 3. Results of Hudi project

Fold	Accuracy	F1 Score	F2 Score
1	0.711	0.711	0.711
2	0.697	0.697	0.742
3	0.816	0.809	0.892
4	0.750	0.749	0.795
5	0.711	0.710	0.753
6	0.737	0.732	0.789
7	0.737	0.735	0.842
8	0.658	0.642	0.747
9	0.724	0.723	0.783
10	0.789	0.780	0.860

The performance metrics of the model declare it to be effective in dealing with the classification task. An F1 Score of 0.7060 reflects a well-balanced trade-off between precision (0.6964) and recall (0.8209). The calculated score, 0.7440, for the F2 Score gives more weight to recall, a good measure for when someone wants to be very sure of finding true positives. The ROC-AUC test showed that the model can distinguish well between the classes, with a score of 0.7941. It includes 2,304 negative instances and 380 positive ones so that the model could perform well on an imbalanced dataset. The optimum parameters achieved were 20 neighbors, a power parameter of 2, and distance-based weights. The best F2 Score obtained with those parameters is 0.7904, a good example showing that the model emphasizes the correct recall with a proper level of precision.

Table 4. Results of Incubator-Celeborn project

Fold	Accuracy	F1 Score	F2 Score
1	0.857	0.844	0.909
2	0.714	0.708	0.652
3	0.500	0.333	0.714
4	0.667	0.625	0.714
5	0.833	0.829	0.938
6	0.833	0.778	0.952
7	0.667	0.667	0.667
8	0.833	0.829	0.909
9	0.500	0.486	0.526
10	0.833	0.829	0.938

It is a good model performance, as reflected by the F1 Score of 0.7216, which presents a balance between a Precision of 0.690 and a Recall of 0.8517. The model indicates high recall, meaning that this model identifies true positives very well but at the cost of low precision. That is, because the F2 score is equal to 0.8219, it is also a reasonably large value for recall. The ROC-AUC Score of 0.7957 reveals a solid ability to distinguish between classes. The model was tested on 283 negative instances and 31 positive instances, hence performing well with a relatively imbalanced dataset. The optimal set parameters chosen are five neighbors, two as the power parameter, and weights according to distance. The best F2 Score obtained with these parameters is 0.8273, meaning the model is good at balancing recall and precision while keeping precision reasonably high.

Table 5. Results of ShardingSphere-ElasticJobproject

Fold	Accuracy	F1 Score	F2 Score
1	0.780	0.772	0.847
2	0.707	0.706	0.783
3	0.659	0.653	0.800
4	0.756	0.749	0.754
5	0.805	0.790	0.887
6	0.610	0.610	0.706
7	0.878	0.878	0.865
8	0.659	0.658	0.833
9	0.725	0.716	0.818
10	0.700	0.670	0.752

Model performance is evaluated with test data composed of 370 negative and 204 positive instances. The best model was found with 18 neighbors, a power parameter of 1, and distance-based weights for the best F2 Score of 0.8273, at which recall was maximized. F1 is 0.6861, and by this value, we can say it balances both precision, which is equal to 0.6928, and recall, which is equal to 0.8528. The F2 Score has a higher value at 0.7485. In this case, it emphasizes the higher level of recall, as in the high-recall model. The substantial ROC-AUC Score value of 0.8376 implies it can differentiate very well between classes; hence, it will make classifications robust and reliable.

Comments on Results

The Bookkeeper dataset indicates decently balanced indices, where the number of 'No' indices is much more than that of 'Yes.' For this set of optimization parameters, the model hyperparameters were `n_neighbors=11`, `p=2`, and `weights=distance`, which provides a best F2 score of 0.7875. F2 scoring weighs recall more heavily and, therefore would be suitable for when false negatives are more important. The accuracy varies from 0.619 to 0.810 among folds and the F1 and F2 scores of the model record balanced values in terms of precision and recall. A ROC-AUC score of 0.7672 indicates that the model is good at separating classes.

The parameters $n_neighbors = 20$, $p = 2$, and $weights = distance$ optimized the Hudi dataset best, giving the best F2 score of 0.7904.

In this dataset, same as for Bookkeeper, the 'No' indices outnumbered the 'Yes' indices. The model gave an excellent consistent performance over the folds with 0.658 to 0.816 accuracy. With a value of 0.8209 for recall and a high ROC-AUC score of 0.7941, it can be interpreted that the model is likely to be very good in identifying positive cases but remains at a good balance between sensitivity and specificity. The Incubator-Celeborn data set was much smaller, with a significantly more imbalanced distribution between the 'No' and 'Yes' indices. Tuning the model parameters for this data set, at the end to $n_neighbors = 5$, $p = 2$, $weights = distance$, the best F2 score obtained was 0.8273—the best out of all the data sets. Nonetheless, fold performance varies: the model is good at recall (0.8517) and ROC-AUC (0.7957). This means it is a very effective model in correctly identifying 'Yes' indices, though the precision is a bit lower at 0.690, indicating somewhat more false positives.

This dataset showed a much better balance in the number of 'No' and 'Yes' indices through the ShardingSphere-ElasticJob model. The best score is 0.8273 with $n_neighbors = 18$, $p = 1$, $weights = distance$. There was a relatively stable performance across folds, with accuracy ranging between 0.610 and 0.878. The model showed high recall—0.8528—and an average ROC-AUC score of 0.8376, signaling good model performance regarding class separation. Good balanced precision (0.6928) and recall make this model reliable and robust in different scenarios.

Overall, the model generalized well across different datasets with varying distributions. Bookkeeper: Performed with good balance, shown through an F2 score to be appropriate for recall-focused tasks. Hudi: Good performance consistently is reflected by a high recall score and ROC-AUC score, making it robust for identifying positive cases.

Incubator-Celeborn: Highest F2 and recall with high variability across folds. ShardingSphere-ElasticJob: Best ROC-AUC, score balanced with precision and recall, suggesting the most stable performance overall in different scenarios. Such comments would help to comprehend the model's effectiveness using a particular dataset and hence facilitate one to choose the dataset required under some prerequisites and priorities.

6 Conclusion

6.1 Summary and conclusions

In this thesis, the prioritization of technical debt repayment is carried out through a human-centered approach, and one considers that implications for the use of automated analysis tools and machine learning techniques in further improving the same prioritization and debt repayment process will have to be designed with the development team's needs and priorities at the forefront. This shows that data quality and integrity are essential considerations in developing predictive models that are reliable and accurate. In this regard, the Python models earlier were used for consistency and stability given development on the transition front. However, the latest model with cross-validation has already shown better performance.

6.2 Limitations of the Research

This study—though pioneering and promising—invariably suffers from several limitations. Realizing such limitations, it is essential to put the findings into a better perspective and understand the scope of applicability of the results. Some fundamental limitations identified in the current research include:

6.3 Interpretability Challenges:

SHAP and LIME are designed to improve model interpretability, but they present their challenges. SHAP values are not simple to interpret for a stakeholder, such as one not necessarily well versed in machine learning. The LIME method, similar to the SHAP method, might give local interpretations, but models have a problem capturing global features because this can lead to possible model misrepresentations.

6.4 Data Limitations:

The study leans heavily on the quality and comprehensiveness of the data sets used. The issues of imbalanced data and duplicates have been taken care of in the pre-processing, but still play a considerable challenge. Handling the imbalance of the data

remains a critical part because it can influence the generalization and accuracy of the models.

6.4.1 Generalization of Models:

The results that have been drawn are strictly based on the particular datasets and models that do not ensure the generalizability of the results for some other datasets or different kinds of machine learning models.

6.4.2 Overfitting:

On the other hand, using the same project for training and testing can make the application model insensitive to some raw data or unimportant patterns, hence overfitting and reducing the generalization ability to new data.

6.4.3 Scalability:

Techniques such as LIME are scalable to different datasets but hit limitations when applied to huge datasets. One of the significant drawbacks, in the case of small datasets, is to give importance to the features as insignificant because of the lesser samples or data points; this further impairs the feature scalability and effectiveness of the method.

6.4.4 Future Improvements Required:

Active research and development are going on to address the existing limitations of these methods. Computational efficiency, interpretability, and general handling of diverse datasets are the areas in which many additional contributions could be made in future research.

In summary, although this work has made some breakthroughs and contributed a lot to the model interpretability and technical debt management study, some limitations should be considered. Future studies must overcome such constraints to make the approach effective, scalable, and generalizable. This will, therefore, not only involve technical improvements but also interdisciplinary efforts to make these methods more accessible and interpretable to a broader range of stakeholders.

6.5 Future Expansion

In the future, work on the approach should further relax these limitations to make it more effective, scalable, and generalizable. This would require not only technical improvements but also interdisciplinary efforts to make such methods more accessible and interpretable to a broader range of stakeholders. This work, in sum, contributed to model interpretability and technical debt management in good measure but still has limitations worth noting. Future research should address these constraints to improve further the effectiveness, scalability, and generalizability of the approach.

References

- Adadi, A., & Berrada, M. (2018). Peeking inside the black-box: a survey on explainable artificial intelligence (XAI). *IEEE access*, 6, 52138-52160.
- Alharbi, M., & Alshayeb, M. (2024). A Comparative Study of Automated Refactoring Tools. *IEEE Access*.
- AlOmar, E. A., AlOmar, S. A., & Mkaouer, M. W. (2023, May). On the use of static analysis to engage students with software quality improvement: An experience with PMD. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)* (pp. 179-191). IEEE.
- Ayewah, N., Pugh, W., Hovemeyer, D., Morgenthaler, J. D., & Penix, J. (2008). Using static analysis to find bugs. *IEEE software*, 25(5), 22-29.
- Beck, K. (2000). *Extreme programming explained: embrace change*. Addison-Wesley Professional.
- Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (Vol. 4, No. 4, p. 738). New York: Springer.
- Canal, C., Poizat, P., & Salaün, G. (2008). Model-based adaptation of behavioral mismatching components. *IEEE Transactions on Software Engineering*, 34(4), 546-563.
- Christoph, M. (2020). *Interpretable machine learning: A guide for making black box models explainable*. Leanpub.
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1), 21-27.
- Dig, D., Johnson, R., Tip, F., De Moor, O., Becicka, J., Griswold, W. G., & Keller, M. (2008). Refactoring Tools: Report on the 1st Workshop WRT at ECOOP 2007. In *Object-Oriented Technology. ECOOP 2007 Workshop Reader: ECOOP 2007 Workshops*, Berlin, Germany, July 30-31, 2007, Final Reports 21 (pp. 193-202). Springer Berlin Heidelberg.
- Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
- Eclipse Foundation. (n.d.). Eclipse Java Development Tools (JDT). Retrieved June 30, 2024, from <https://www.eclipse.org/jdt/>
- Fix, E. (1985). *Discriminatory analysis: nonparametric discrimination, consistency properties* (Vol. 1). USAF school of Aviation Medicine.

Fowler, M. (2004). Inversion of control containers and the dependency injection pattern. <http://www.martinfowler.com/articles/injection.html>.

Fowler, M. (2018). Refactoring: improving the design of existing code. Addison-Wesley Professional.

Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.

Google. (n.d.). CodePro Analytix. Retrieved June 30, 2024, from <https://developers.google.com/eclipse/docs/codepro>

Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., & Pedreschi, D. (2018). A survey of methods for explaining black box models. *ACM computing surveys (CSUR)*, 51(5), 1-42.

Holtmann, J., Bernijazov, R., Meyer, M., Schmelter, D., & Tschirner, C. (2016). Integrated and iterative systems engineering and software requirements engineering for technical systems. *Journal of Software: Evolution and Process*, 28(9), 722-743.

Hutton, D. M. (2009). Clean code: a handbook of agile software craftsmanship. *Kybernetes*, 38(6), 1035-1035.

Johnson, R., Hoeller, J., Arendsen, A., & Thomas, R. (2009). Professional Java development with the Spring framework. John Wiley & Sons.

Kamiya, T. (2015, March). An execution-semantic and content-and-context-based code-clone detection and analysis. In 2015 IEEE 9th international workshop on software clones (IWSC) (pp. 1-7). IEEE.

Kerpicci, M., Prvulovic, M., & Zajić, A. (2022). A hierarchical approach for multiple periodicity detection in software code analysis. *IEEE Access*, 10, 106936-106945.

Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C., Loingtier, J. M., & Irwin, J. (1997). Aspect-oriented programming. In *ECOOP'97—Object-Oriented Programming: 11th European Conference Jyväskylä, Finland, June 9–13, 1997 Proceedings 11* (pp. 220-242). Springer Berlin Heidelberg.

Kruchten, P., Nord, R. L., & Ozkaya, I. (2012). Technical debt: From metaphor to theory and practice. *Ieee software*, 29(6), 18-21.

Long, J., & Bastani, K. (2017). Cloud Native Java: Designing Resilient Systems with Spring Boot, Spring Cloud, and Cloud Foundry. "O'Reilly Media, Inc."

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30.

Lundberg, S. M., Erion, G. G., & Lee, S. I. (2018). Consistent individualized feature attribution for tree ensembles. *arXiv preprint arXiv:1802.03888*.

Mens, T., & Tourwé, T. (2004). A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2), 126-139.

Mikki, S., Clauzier, S., & Antar, Y. (2019). A correlation theory of antenna directivity with applications to superdirective arrays. *IEEE Antennas and Wireless Propagation Letters*, 18(5), 811-815.

Mosavi, S. A. B. (2021). Finding optimal point features in transient multivariate.

Murphy-Hill, E., & Black, A. P. (2008). Refactoring tools: Fitness for purpose. *IEEE software*, 25(5), 38-44.

Murphy, K. P. (2012). *Machine learning: a probabilistic perspective*. MIT press.

Oliveira, M., Mota, A., & Reis, J. (2024). An automated Android exploratory testing based on code evolution coverage.

Powers, D. M. (2020). Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *arXiv preprint arXiv:2010.16061*.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016, August). " Why should i trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1135-1144).

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PloS one*, 10(3), e0118432.

SonarQube. (n.d.). SonarQube. Retrieved June 11, 2024, from <https://www.sonarqube.org/>

Tan, L., & Bockisch, C. (2019). A Survey of Refactoring Detection Tools. In *Software Engineering (Workshops)* (pp. 100-105).

Thompson, S., & Li, H. (2013). Refactoring tools for functional languages. *Journal of Functional Programming*, 23(3), 293-350.

Tom, E., Aurum, A., & Vidgen, R. (2013). An exploration of technical debt. *Journal of Systems and Software*, 86(6), 1498-1516.

Tsantalis, N., Chaikalis, T., & Chatzigeorgiou, A. (2008, April). JDeodorant: Identification and removal of type-checking bad smells. In 2008 12th European conference on software maintenance and reengineering (pp. 329-331). IEEE.

Tsantalis, N., Ketkar, A., & Dig, D. (2020). RefactoringMiner 2.0. IEEE Transactions on Software Engineering, 48(3), 930-950.

Understand. (n.d.). Understand: Static Code Analysis. Retrieved June 30, 2024, from <https://scitools.com/>

Walls, C. (2015). Spring Boot in action. Simon and Schuster.

Wang, B. Frequency response indicator for web form.

White, J., & Wheeler, W. (2013). Spring in practice. Simon and Schuster.

Wiegand, J. (2004). Eclipse: A platform for integrating development tools. IBM Systems Journal, 43(2), 371-383.